COMP3003: Report (Reinforcement Learning & Classification)

Contents

1. Task (3) Reinforcement Learning

Reinforcement learning: a goal-directed (for the highest reward) consequence-driven (feedback-oriented) approach wherein an agent interacts with an unknown environment through trial and observation; the agent 1) observes a reward, then 2) adjusts its behaviour to achieve the highest reward. The goal is to maximise cumulative reward; to achieve this the agent will be committed to the *optimal policy* to attain the highest long-term return.

Unlike supervised learning (with labelled input-output pairs) or unsupervised learning (that discovers patterns without labels), RL provides only a scalar reward signal as feedback. Kaelbling, Littman and Moore (1996, p. 237) characterise this as learning "through trial-and-error interactions with a dynamic environment."

Task (3) applies these principles to a 5-state MDP shown in *Figure 1* with deterministic transitions and a single high-reward transition ($S_4 \rightarrow S_5$, $r = 10$). I will derive the optimal policy, compute $V^*(S_5)$ symbolically in terms of γ, and trace Q-learning under a greedy policy, thereby revealing the exploration-exploitation trade-off when the agent becomes trapped in a suboptimal loop.

1.1 Overview

Considering the following model in Figure 1 with 5 states with *Right* and *Left* actions, and $\gamma = 0.8$:

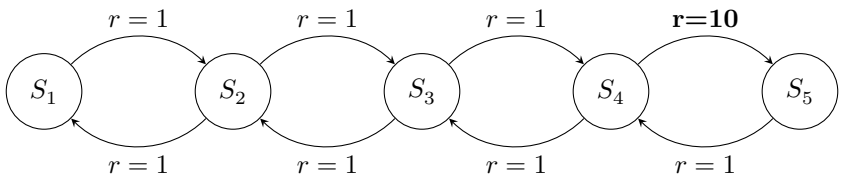

Figure 1: Markov Decision Process with States S_1 to S_5, with a high-reward transition at $S_4 \rightarrow S_5$.

This figure shows a Markovian decision process (Bellman, 1957, p. 684) where decisions at each stage "transform the system from its present state into another state" via a transition matrix; basically, every action taken by the agent reshapes the current situation into a new one, and the transition matrix is simply the rulebook encoding which outcomes follow from which choices. In this environment (the playground), an agent (the decision-maker) operates to maximise cumulative rewards, a goal formally described as optimising the "infinite-horizon discounted model" (Kaelbling, Littman and Moore, 1996, p. 240). At S_1, the agent can only go to S_2, and at S_5, can only go to S_4.

Based on the diagram and the description, I will proceed with this assumption: each individual action yields a reward of `r=1`, with the single exception of the $S_4 \rightarrow S_5$ transition, which yields `r=10`. Similarly to the Set Exercises, I will answer each question in the following manner: 1) Answer the first question with a detailed, step-by-step process, explaining my reasoning and the concepts involved; 2) answer the subsequent questions, by providing more concise answers, focusing on the key points and results without repeating the first question's detailed explanations.

In many cases, MDPs contain negative-reward transitions (e.g., -1) to punish agents for taking longer paths. However, in this MDP, it is a continuous task (cyclic) without a terminal state. Kaelbling, Littman and Moore (1996, p. 240) explain that the discounted model uses a factor γ as a "mathematical trick to bound the infinite sum". Thus, there is no terminal state to reach; if the rewards were -1, the agent would be stuck in an infinite loop losing value without being able to recover. The coursework is therefore designed in such a way that the problem is maximising yield from the $S_4 \rightarrow S_5$ transition, rather than minimising loss/pain.

1.2. Describe the optimal policy for the Markov Decision Process (MDP).

To properly deconstruct the first question (i.e. What is the **optimal policy** for this **MDP**), I will first explain what a policy *is* in the context of MDPs, and then I will analyse the MDP to derive the optimal policy for each state.

1.2.1 What is a Policy?

A policy ($\pi(s)$): the strategy an agent employs to decide which action to take to maximise long-term discounted rewards; it is essentially a behavioural script wherein the agent makes a decision at each state. An optimal policy ($\pi^*(s)$) is therefore one that yields the *maximum* possible expected return, or as Kaelbling, Littman and Moore (1996, p. 248) define it, the "expected infinite discounted sum of reward that the agent will gain if it starts in that state and executes the optimal policy".

Within the context of this linear 5-state chain *Figure 1*, a policy ($\pi(s)$) encourages the agent's directional choice at each node: retreat leftward toward S_1 or continue rightward toward the gainful $S_4 \rightarrow S_5$ transition. An optimal policy $\pi^*(s)$ recognises that the endless $S_4 \rightarrow S_5$ loop, regardless of appearing trapped, will yield superior long-term returns (harvesting $r = 10$ repeatedly) compared to the modest $r = 1$ rewards elsewhere.

1.2.2 Understanding the Markov Property

The MDP follows the Markov property: essentially, forget the past and how you arrived here (the current state), and then make a decision. This means the probability of transitioning to the next state depends only on the current state and action. As described by Kaelbling, Littman and Moore (1996, p. 248) when they state that the model is Markov "if the state transitions are independent of any previous environment states or agent actions".

1.2.3 Role of Discount Factor (γ)

Regarding the discount factor, $\gamma = 0.8$ serves a crucial purpose: it is a hyperparameter that prevents infinite agentic maximisation, and it discourages the agent from going on forever. It essentially makes *distant* rewards less valuable thereby ensuring convergence even in MDPs with back-and-forth transitions (cyclic) such as this'; close to 1 = long-term rewards are valued highly whereas close to 0 = short-term rewards are better.

1.2.4. Analysing the Optimal Policy for Each State

To derive the optimal policy, I will analyse each state's available actions and determine which action yields the highest long-term discounted reward, thereby identifying the best action to take in each state.

1.2.4.1 State S_4: The Critical Decision Point

State S_4 presents the agent with two choices of actions:

- **Right** $\rightarrow S_5$: agent receives an *immediate reward* of $r = 10$
- **Left** $\rightarrow S_3$: agent immediately receives reward: $r = 1$

To determine optimality, I will apply the Bellman optimality equation. Kaelbling, Littman and Moore (1996, p. 248) define the optimal value function as:

$$V^*(s) = \max_a \left(R(s,a) + \gamma \sum_{s' \in \mathcal{S}} T(s,a,s') V^*(s') \right)$$

wherein $T(s,a,s')$ denotes "the probability of making a transition from state s to state s' using action a", essentially equivalent to the course-material notation of $P(s'|s,a)$. The authors' formulation represents "the expected instantaneous reward plus the expected discounted value of the next state, using the best available action." Given V^*, the optimal policy follows:

$$\pi^*(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right]$$

In this MDP, transitions are *deterministic*, i.e., each of the transition probabilities in the Markov chain individually equal one, thus:

- Choosing *Right*: $Q^*(S_4, \text{Right}) = 10 + 0.8 \cdot V^*(S_5)$
- Choosing *Left*: $Q^*(S_4, \text{Left}) = 1 + 0.8 \cdot V^*(S_3)$

To compare these, I therefore determine the relationship between $V^*(S_5)$ and $V^*(S_3)$. From S_5, the only action is *Left* $\rightarrow S_4$ with $r = 1$:

$$V^*(S_5) = 1 + 0.8 \cdot V^*(S_4)$$

From S_3, the optimal action is *Right* $\rightarrow S_4$, also with $r = 1$:

$$V^*(S_3) = 1 + 0.8 \cdot V^*(S_4)$$

$V^*(S_5)$ therefore equals $V^*(S_3)$, meaning both actions from S_4 lead to states with identical future value. The decision thus reduces to comparing immediate rewards: since $10 > 1$, *Right* is unequivocally superior.

Therefore: $\pi^*(S_4) = \text{Right}$

1.2.4.2 States S_1, S_2, and S_3: Path to the Reward

- S_1: only action is *Right* $\rightarrow S_2$; thus $\pi^*(S_1) = \text{Right}$
- S_2: Comparing actions:
  - *Right* $\rightarrow S_3$; moves closer to S_4 (and its $r = 10$ transition-reward),
  - *Left* $\rightarrow S_1$; moves away from S_4, i.e. the high-reward state. Since $V^*(S_3) > V^*(S_1)$ due to the closer proximity to the high-$r = 10$ reward, $\pi^*(S_2) = \text{Right}$
- S_3: Comparing actions:
  - *Right* $\rightarrow S_4$; directly leads to the high-reward $r = 10$ transition,
  - *Left* $\rightarrow S_2$; moves away from S_4 (high-reward state). Since $V^*(S_4) > V^*(S_2)$, $\pi^*(S_3) = \text{Right}$

1.2.4.3 State S_5: Return-path State

- S_5: only Left is available $\rightarrow S_4$; thus $\pi^*(S_5) = \text{Left}$

1.2.4.4 Summary

The optimal policy (π^*) for this MDP is:

- $\pi^*(S_1) = \text{Right}$ (forced starting action)
- $\pi^*(S_2) = \text{Right}$ (towards high reward)
- $\pi^*(S_3) = \text{Right}$ (towards high reward)
- $\pi^*(S_4) = \text{Right}$ (captures $r = 10$)
- $\pi^*(S_5) = \text{Left}$ (forced return action)

This creates an optimal trajectory: $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \rightarrow S_5$, after which the agent falls into the $S_4 \rightarrow S_5 \rightarrow ...$ never-ending cycle, forever harvesting the $r = 10$ reward. The optimal policy represents the most direct route to repeatedly capture the high-reward transition, analogous to walking linearly to a door rather than circling the room. This policy is optimal because it maximises the long-term discounted reward by ensuring the agent consistently reaches the high-reward transition (Kaelbling, Littman and Moore, 1996, p. 248). The discount factor $\gamma = 0.8$ ensures convergence despite the infinite $S_4 \leftrightarrow S_5$ cycle.

1.3 Describe V∗(S5) (in terms of γ and not state values)

The value function ($V^*(s)$) represents the expected value i.e. the reward gained from proceeding *forward* from this state, following the optimal policy (π^*, wherein the star denotes optimality) (Kaelbling, Littman and Moore, 1996, p. 248).

Computing $V^*(S_5)$ requires understanding the recursive nature of the Bellman equation. From S_5, the *only* action is clearly Left $\rightarrow S_4$ with the low reward of $r = 1$. You are looking ahead in time to determine which one of these is the maximum, and can trace the optimal future path using this specific Bellman equation:

$$V^*(S_5) = 1 + \gamma \cdot V^*(S_4) \tag{1}$$

Eq. 1 states that the value at S_5 equals immediate reward (1) plus the discounted value of next-state S_4, i.e. $\gamma \cdot V^*(S_4)$

Because the optimal policy creates a back-and-forth cycle between S_4 and S_5, wherein the agent can be trapped in this loop indefinitely whilst still productively harvesting rewards, we now know:

$$V^*(S_4) = 10 + \gamma \cdot V^*(S_5) \quad \text{when } r = 10 \tag{2}$$

Eq. 2 states that from S_4, going Right to S_5 gives an immediate reward of 10 plus the discounted value of S_5

The two equations simultaneously equate in a coupled system wherein each state's value depends recursively on the other's:

$$V^*(S_5) = 1 + \gamma \cdot V^*(S_4) \quad \text{(from Eq. 1)}$$
$$V^*(S_4) = 10 + \gamma \cdot V^*(S_5) \quad \text{(from Eq. 2)}$$

To solve symbolically, I substituted the second equation into the first, thereby eliminating $V^*(S_4)$:

$$V^*(S_5) = 1 + \gamma(10 + \gamma \cdot V^*(S_5))$$

Expanding the brackets and collecting like terms:

$$V^*(S_5) = 1 + 10\gamma + \gamma^2 \cdot V^*(S_5)$$

$$V^*(S_5) - \gamma^2 \cdot V^*(S_5) = 1 + 10\gamma$$

$$V^*(S_5)(1 - \gamma^2) = 1 + 10\gamma$$

The symbolic solution expressed purely in terms of γ is **therefore**:

$$V^*(S_5) = \frac{1 + 10\gamma}{1 - \gamma^2} = \frac{1 + 10\gamma}{(1 - \gamma)(1 + \gamma)}$$

This formula encapsulates the total discounted future reward from state S_5 under the optimal policy, i.e., it quantifies the accumulated value generated by the infinite loop. The denominator $(1 - \gamma^2)$ reflects the geometric series convergence for the infinite S_4-S_5 loop, whilst the numerator $(1+10\gamma)$ captures the immediate and next-step rewards weighted by the discount factor. This ensures an infinite sum converges to a finite value despite the never-ending cycle (Kaelbling, Littman and Moore, 1996, p. 240).

1.4. Consider executing Q-learning on this MDP

"Assume that the Q values for all (state, action) pairs are initialized to 0, that $\alpha = 0.5$, and that Q-learning uses a greedy exploration policy, meaning that it always chooses the action with the maximum Q value. The algorithm breaks ties by choosing Left. What are the first 10 (state, action) pairs if our robot learns using Q- learning and starts in state S3 (e.g., (S3,Left), (S2,Right), (S3,Right),...)?"

Q-learning is fundamentally about the Q-function: a function that takes the state and the action and gives a value defined by Watkins and Dayan (1992, p. 280). The brief specifies a "greedy exploration policy, meaning that it always chooses the action with the maximum Q value"; since no epsilon (ε) is provided, this constitutes pure exploitation wherein the agent deterministically follows the highest Q-value estimate rather than occasionally sampling alternative actions. With this purely exploitative policy and Left tie-break in this MDP, the agent can become trapped in a loop, i.e., it essentially ends up spinning; Kaelbling, Littman and Moore (1996, p. 254) acknowledge that "during learning, however, there is a difficult exploitation versus exploration trade-off to be made". Thus, the update rule, for the Q-values, is as follows:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big]$$

wherein the following-specified parameters are given: 1) the agent begins in state S_3; all Q-values are initialised to 0; the reward r is taken directly from each transition; 2) the *next* state (s'), follows deterministically from the diagram; $\gamma = 0.8$ applies the discounting; $\alpha = 0.5$ updates the correction, thus the greedy policy goes Left for tie-breaks. The bracketed term is the *temporal-difference (TD) error*: the discrepancy between the target estimate $r + \gamma \max_{a'} Q(s',a')$ and the current estimate $Q(s,a)$. The learning rate α then scales how aggressively this correction is applied.

It is important to note: with all Q-values at zero, every initial decision ties; the agent therefore blindly goes Left until some Q-value becomes non-zero and breaks the symmetry.

1.4.1 Steps 1-4 Trajectory

1. At S_3, both actions: $Q(S_3, L)$ and $Q(S_3, R)$, equal 0. The *greedy policy* will select the action with maximum Q-value, thereby resolving what Kaelbling, Littman and Moore (1996, p. 243) call the "fundamental tradeoff between exploitation and exploration" by sending the agent entirely (greedily) toward exploitation. With tied values, the tie-break rule selects Left.

- *Action:* Left $\rightarrow S_2$ gaining reward $r = 1$
- *Update:* $Q(S_3, L) \leftarrow 0 + 0.5[1 + 0.8 \cdot \max(0,0) - 0] = 0.5$
- **Pair 1** is therefore: $(S_3, Left)$

2. At S_2, both actions, $Q(S_2, L) = 0$ and $Q(S_2, R) = 0$, tie-break thus go Left $\rightarrow S_1$ with $r = 1$:

- *Action:* Left $\rightarrow S_1$; receives reward $r = 1$
- *Update:* $Q(S_2, L) \leftarrow 0 + 0.5[1 + 0.8 \cdot \max(0,0) - 0] = 0.5$
- **Pair 2** is therefore: $(S_2, Left)$

3. At S_1, only action, Right, is available per *Figure 1*:

- *Action:* Right $\rightarrow S_2$ receiving reward $r = 1$
- *Update:* Having arrived at S_2, the agent estimates the future value via $\max_{a'} Q(S_2, a') = \max(0.5, 0) = 0.5$. Thus:

$$Q(S_1, R) \leftarrow 0 + 0.5[1 + 0.8(0.5) - 0] = 0.7$$

- **Pair 3** is therefore: $(S_1, Right)$

4. At S_2, now $Q(S_2, L) = 0.5 > Q(S_2, R) = 0$:

- *Action:* Left $\rightarrow S_1$, receiving $r = 1$

- *Update:* From S_1, the only action has $Q(S_1, R) = 0.7$, so $\max_{a'} Q(S_1, a') = 0.7$. Thus: $Q(S_2, L) \leftarrow 0.5 + 0.5[1 + 0.8(0.7) - 0.5] = 1.03$
- **Pair 4** is therefore: (S_2, Left)

1.4.2 Emergent Oscillatory Pattern (Steps 5-10)

From Step 4 onwards, a self-reinforcing pattern emerges: at S_2, $Q(S_2, L) > Q(S_2, R) = 0$ forces Left; at S_1, only Right exists. The agent oscillates between these two states, with each Q-value bootstrapping from the other's increasing estimate.

Table 1: Q-value updated during oscillation (cycling) between S_1 and S_2

Step	**State**	**Action**	r	$\max_{a'} Q(s', a')$	**Updated Q-value**
5	S_1	Right	1	$Q(S_2, L) = 1.03$	$Q(S_1, R) = 1.262$
6	S_2	Left	1	$Q(S_1, R) = 1.262$	$Q(S_2, L) = 1.520$
7	S_1	Right	1	$Q(S_2, L) = 1.520$	$Q(S_1, R) = 1.739$
8	S_2	Left	1	$Q(S_1, R) = 1.739$	$Q(S_2, L) = 1.956$
9	S_1	Right	1	$Q(S_2, L) = 1.956$	$Q(S_1, R) = 2.152$
10	S_2	Left	1	$Q(S_1, R) = 2.152$	$Q(S_2, L) = 2.339$

This exploitative behaviour exemplifies the failure of greedy policies. As Kaelbling, Littman and Moore (1996, p. 246) describe, the "suboptimal action will always be picked, leaving the true optimal action starved of data and its superiority never discovered". The high-reward $S_4 \rightarrow S_5$ transition remains permanently unknown precisely because (S_3, Right) is never sampled.

Sanity check: The Q-value increments diminish through each cycle (Step 4: +0.53, Step 6: +0.49, Step 8: +0.44, Step 10: +0.38). This geometric decay occurs because the Bellman operator is a γ-contraction: successive updates shrink the gap between current Q-values and Q^* by factor γ at each iteration (Tsitsiklis, 1994, p. 199). With $\gamma = 0.8$, the observed reduction, per cycle, aligns with this theory.

Answer: The first 10 (state, action) pairs are:

1. (S_3, Left) - tie-break (both actions equal 0)
2. (S_2, Left) - tie-break
3. (S_1, Right) - forced (only action)
4. (S_2, Left) - greedy
5. (S_1, Right) - forced
6. (S_2, Left) - greedy
7. (S_1, Right) - forced
8. (S_2, Left) - greedy
9. (S_1, Right) - forced
10. (S_2, Left) - greedy

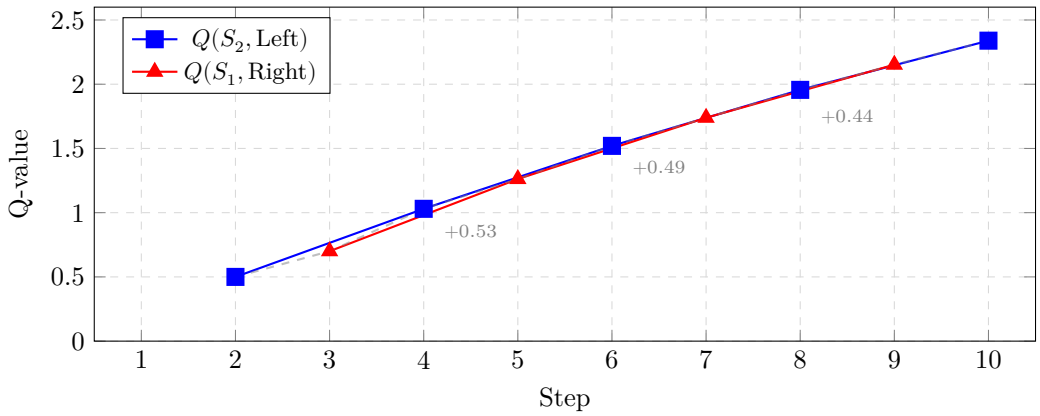

Figure 2: Q-value bootstrapping during the $S_1 \leftrightarrow S_2$ oscillation. The dashed line traces the alternating updates wherein each state's Q-value *bootstraps* from the other's increasing estimate (each uses the other's guess to update itself), thereby creating a self-reinforcing cycle. The diminishing increments align with Tsitsiklis's (1994, p. 199) result that the discounted Bellman operator is a contraction mapping (each update shrinks the error norm) with respect to $\|\cdot\|_\infty$; this ensures bounded convergence despite the infinite loop.

1.5. Discussing the implications of using a greedy policy (pure exploitation) in Q-learning.

When considering the implications of a greedy exploration policy in Q-learning, we can first note the $S_1 \leftrightarrow S_2$ oscillation observed, i.e., the agent becomes trapped in a loop indefinitely, discussed in Section 1.4.2 and it exemplifies a fundamental failure mode of purely greedy exploration policies… Watkins and Dayan (1992, p. 279) mathematically proved that Q-learning converges to optimal action-values: "with probability 1 so long as all actions are repeatedly sampled in all states." However this condition is violated in this coursework's trace: after Step 1, $Q(S_3, \text{Left}) = 0.5 > Q(S_3, \text{Right}) = 0$, the greedy policy is caused to assign $\pi(S_3, \text{Right}) = 0$ permanently. Consequently, states S_4 and S_5, and importantly the high-reward $S_4 \rightarrow S_5$ transition, are never explored and thus discovered.

This behaviour demonstrates what Kaelbling, Littman and Moore (1996, p. 246) identify as the fundamental limitation of greedy policies: the agent's knowledge becomes concentrated within the exploited trajectory whilst remaining absent for unexplored state-action pairs.

1.5.1 Deterministic Trap Permanence

This MDP's deterministic transitions ($T(s'|s, a) = 1$) exacerbate this failure. In stochastic environments, transitions sometimes expose the agent to unexplored regions even under greedy policies. Kaelbling, Littman and Moore (1996, p. 271) illustrate this with backgammon, a two-player board game where dice rolls determine available moves, noting that "the state transitions are sufficiently stochastic that independent of the policy, all states will occasionally be visited". However, deterministic dynamics preclude this mechanism. Furthermore, this MDP lacks a terminal state; the system loops indefinitely, meaning the agent cannot escape through episode termination. Once the agent commits to the $S_1 \leftrightarrow S_2$ loop, no escape is possible and thus the trap is permanent.

1.5.2 The Exploration-Exploitation Issue

This result demonstrates what Kaelbling, Littman and Moore (1996, p. 243) classify as an exploration-exploitation trade off: purely exploitative behaviour (greedy selection) prevents discovery of a better strategy; this greedy behaviour is determined by setting the exploration parameter, i.e. epsilon (ε), to zero. The standard solution to this dilemma is ε-greedy exploration, wherein the agent selects a random action with probability $\varepsilon > 0$ (Kaelbling, Littman and Moore, 1996, p. 246). This ensures $\pi(s, a) > 0$ for all state-action

pairs, also satisfying Watkins and Dayan's convergence requirement (1992, p. 279). Even modest exploration ($\varepsilon = 0.1$) would eventually sample (S_3, Right), initiating the discovery of the optimal $S_4 \leftrightarrow S_5$ cycle. The challenge of exploration in deterministic traps is not unique to this simple MDP; it remains a critical issue in modern applications. For instance, Mnih et al. (2015, p. 7 *of the .pdf) utilised an ε-greedy strategy in their Deep Q-Network (DQN), specifically: "following a behaviour distribution that ensures adequate exploration of the state space". This prevented the agent from continuing to move whilst failing to improve (i.e. stagnating) in suboptimal loops similar to the $S_1 \leftrightarrow S_2$ cycle observed here.

1.5.3 Comparison with Literature

The observed-trap behaviour aligns with Kaelbling, Littman and Moore's (1996, p. 246) analysis regarding data starvation; because the greedy policy never samples the alternative action, the optimal path remains permanently invisible to the agent.

The empirical result, converging Q-values within an exploited subspace whilst ignoring the optimal policy, aligns concretely with their prediction; furthermore, Tsitsiklis (1994, p. 199) emphasises that convergence requires "every state-action pair (i,u) is simulated an infinite number of times", a condition this greedy policy clearly violates for pairs such as (S3, Right), (S4, Left), (S4, Right), and all S5 pairs.

2. Task (4) Classification (2350-word; 10% allowance)

2.1. Abstract (<150*1.1 words)

Polycystic-ovary syndrome (PCOS) has a prevalence of 6-12% in reproductive-age women (Lizneva et al., 2016, *Abstract*); nonetheless, diagnosis remains difficult due to complex multi-feature interactions across 41 clinical features that are well-suited to machine learning classifiers that can model multi-feature interactions.

2.1.1 Methodology

This coursework comparatively evaluates neural network and Gaussian Naive Bayes classifiers on 541 patient records (samples) with 41 clinical features, employing fold-wise z-score normalisation to prevent data leakage, and stratified 10-fold cross-validation experimentally selected for stability-adjusted performance.

2.1.2 Key Results

The neural network (41-25-12-1 architecture) achieved 87.8% accuracy with AUC of 0.945, outperforming Naive Bayes (84.5% accuracy, AUC 0.901) on discrimination metrics; however, Naive Bayes demonstrated superior recall (84.8% versus the NN's 78.7%), indicating higher recall on this classification task despite violated independence assumptions.

2.1.3 Conclusion

Therefore: classifier choice depends on the application's cost function; neural networks maximise discrimination, whereas Naive Bayes maximises recall, suiting contexts where false negatives are penalised more heavily.

2.2. Introduction (<600*1.1 words)

Classification: a supervised learning approach wherein algorithms learn decision boundaries from *labelled* training data-examples to categorise unseen outcomes from unseen instances (Kotsiantis, 2007, p.249). Unlike un-supervised methods e.g. K-means clustering (*Set-exercise Task (2)*), supervised classification exploits known input-*features* → output-*label* mappings to minimise prediction error on testing data.

2.2.1 Neural Networks

Neural networks (NNs) are models inspired by biological neuronal structures in brains, wherein weighted inputs pass through *non-linear* activation functions to enable learning of arbitrarily complex decision boundaries. The universal-approximation theorem establishes the *feedforward* networks (forward-only) with sufficient hidden neurons can approximate any continuous function to arbitrary precision (Hornik, Stinchcombe and White, 1989, *Abstract*). In medical classification contexts, Amato et al. (2013, pp. 48, 52) reports neural networks achieving 99.5% accuracy on breast cancer diagnosis, attributing this to its model's capacity for capturing non-linear interactions between clinical biomarkers. However, this flexibility, thus expressiveness, comes at computational expense: neural networks need iterative gradient-based optimisation through potentially thousands-to-millions of parameters, with performance sensitive (reactive) to hyperparameter configuration. Smith (2017, *Abstract*) noted that "It is known that the learning rate is the most important hyper-parameter to tune for training deep neural networks" as suboptimal settings can prevent convergence completely or trap the model in poor local minima. Bergstra and Bengio (2012, p.281) further highlighted that "randomly chosen trials are more efficient for hyper-parameter optimisation than trials on a grid". Unlike neural networks, Naive Bayes requires minimal hyperparameter configuration, though it imposes stronger distributional assumptions.

2.2.2 Naive Bayes Classification

Naive Bayes classification applies *Bayes' theorem* under an independence assumption: rather than counting exact feature combinations, it multiplies individual feature probabilities:

$$P(C \mid \mathbf{x}) = \frac{P(C) \cdot P(\mathbf{x} \mid C)}{P(\mathbf{x})}$$

Under the *naive* conditional independence assumption that features are independent given the class, the joint likelihood decomposes as:

$$P(\mathbf{x} \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

Substituting into Bayes' theorem yields the classification rule shown below.

Naive Bayes Classification Rule

$$P(C \mid \mathbf{x}) \propto P(C) \prod_{i=1}^{n} P(x_i \mid C) \tag{3}$$

Interpretation: Multiply the prior ($P(C)$), i.e. how prevalent is the class, by the likelihood of each feature given (|) the class, i.e. whichever class gives the highest product wins.

This decomposition yields tractable computation (parameters computed directly, no iteration), requiring only $O(n \cdot k)$ parameters for n features and k classes, versus the exponential parameter growth of full Bayesian network classifiers (Friedman, Geiger and Goldszmidt, 1997, p.136).

The training requires just one traversal of the 541-patient dataset to estimate class-conditional means and variances, with prediction involving low-cost probability multiplication instead of expensive marginalisation. Despite the independence assumption rarely holding in practice, Domingos and Pazzani (1997, p. 113) established that Naive Bayes achieves optimal classification under zero-one loss "as long as the class with the highest estimated probability, $C_X(E)$, is the class with the highest true probability". This theoretical insight explains why Naive Bayes often performs competitively even when features exhibit substantial correlations, as in medical datasets where hormonal markers frequently co-vary.

The choice of likelihood distribution fundamentally impacts Naive Bayes performance. For continuous features, the Gaussian distribution represents the default assumption, modelling each feature as normally

distributed within each class. Alternative distributions include kernel density estimation for multi-modal features, and the multinomial distribution for count data (Manning, Raghavan and Schütze, 2008, p.258). The selection should be guided by empirical examination of feature distributions rather than computational convenience.

This coursework addresses PCOS diagnosis, a condition affecting reproductive-age women wherein various clinical features interact complexly. The dataset comprises 541 patient records with 41 clinical and biochemical features, providing sufficient samples for neural network training whilst remaining tractable for Naive Bayes parameter estimation. The following sections detail the implementation of both classifiers, systematic k-fold selection, and comprehensive performance comparison using discrimination and calibration metrics.

2.3. Implementation (<800*1.1 words)

2.3.1 Dataset Description and Preprocessing

The raw 44-attribute schema was refined to 41 predictive features by excluding non-predictive identifiers: `"Sl. No"` (serial number), `"Patient File No."` (patient identifier), and `"Unnamed: 44"` (empty artefact). Removing these non-informative columns is a critical regularisation step; leaving unique identifiers allows the model to memorise IDs rather than learning generalisable clinical patterns, thus overfitting.

Listing 1: Matlab implementation for non-predictive feature removal

```
colsToRemove = {'Sl. No', 'Patient File No.', 'Unnamed: 44'}; % unimportant predictive values;
    ↪ non-predictive identifiers
fprintf('\nRemoving non-predictive columns:\n');
for i = 1:length(colsToRemove) % for each non-predictive column, if it exists, remove it to
    ↪ prevent memorisation, i.e. when the model learns to associate specific IDs with labels seen
    ↪ during training runs thereby overfitting
   colName = colsToRemove{i};
   if any(strcmp(rawData.Properties.VariableNames, colName))
      rawData(:, colName) = [];
      fprintf(' - Removed: "%s"\n', colName);
   end
end
```

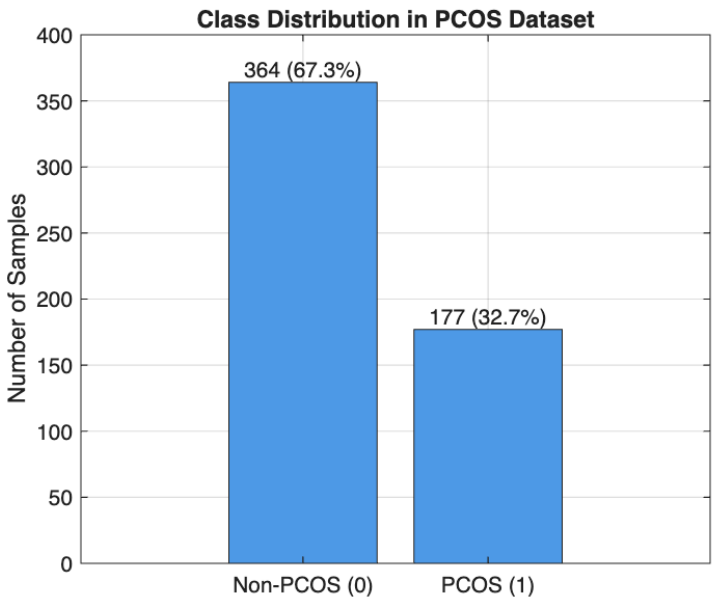

Figure 3: Class distribution in the PCOS dataset.

The class distribution in (*Figure~3*) exhibits moderate imbalance: 177 PCOS-positive cases (32.7%) versus 364 PCOS-negative cases (67.3%), yielding an imbalance ratio of 2.06:1 (~two negative samples per positive sample). This necessitates stratified cross-validation to preserve class proportions across folds.

2.3.2 Preprocessing: Fold-wise Normalisation

Missing values were handled via fold-wise mean imputation exclusively using training statistics; fold-wise z-score normalisation was applied as follows:

Listing 2: Fold-wise preprocessing to prevent data leakage

```
% Important: Impute test set using train means, not test means,
% if test data influenced imputation, model would have indirect knowledge of test distribution
    ↪ (data leakage)
trainMeans = mean(xTrainRaw, 'omitnan'); % omit NaNs when calculating means
xTrain = fillmissing(xTrainRaw, 'constant', trainMeans);
xTest = fillmissing(xTestRaw, 'constant', trainMeans);

mu = mean(xTrain, 1);
sigma = std(xTrain, 0, 1);
sigma(sigma == 0) = 1; % prevent division by zero for constant features
xTrain = (xTrain - mu) ./ sigma; % these lines compute z = (x - mu) / sigma
xTest = (xTest - mu) ./ sigma; % element-wise normalisation of the test set
```

This approach computes normalisation statistics $z = \frac{x-\mu_{\text{train}}}{\sigma_{\text{train}}}$ exclusively from training folds, ensuring clinical measurements with disparate units (e.g., BMI in kg/m^2 versus FSH in mIU/mL) contribute equally to optimisation whilst preventing test data from influencing preprocessing.

The standardisation is vital for the neural network as without it features with large magnitudes (e.g. Prolactin levels) would disproportionately influence the Euclidean error surface or saturate the sigmoid activation functions. Saturation pushes neuron outputs into the flat regions of the curve where gradients approach zero; this induces a regime where the variance of the back-propagated gradient can "vanish or explode" (Glorot & Bengio, 2010, p. 253), thereby stalling weight updates during backpropagation.

2.3.3 Cross-Validation Selection

Three k-fold configurations were evaluated experimentally to determine optimal partitioning. For 541 samples: $k = 5$ yields approximately 108 samples per fold, $k = 10$ yields 54, and $k = 15$ yields 36.

Selection employed a stability-adjusted performance criterion penalising high variance:

$$\text{Score}_k = \bar{A}_k - \underbrace{0.5}_{\text{minimise by half}} \cdot \sigma_k \tag{4}$$

wherein $\bar{A}_k$ denotes mean accuracy whilst σ_k denotes standard deviation across folds.

Table 2: K-fold comparison results during selection phase

k	Neural Network	Naive Bayes
5	89.3% ± 1.9%	83.0% ± 4.8%
10	89.8% ± 2.9%	83.2% ± 4.5%
15	86.7% ± 5.6%	85.0% ± 4.3%

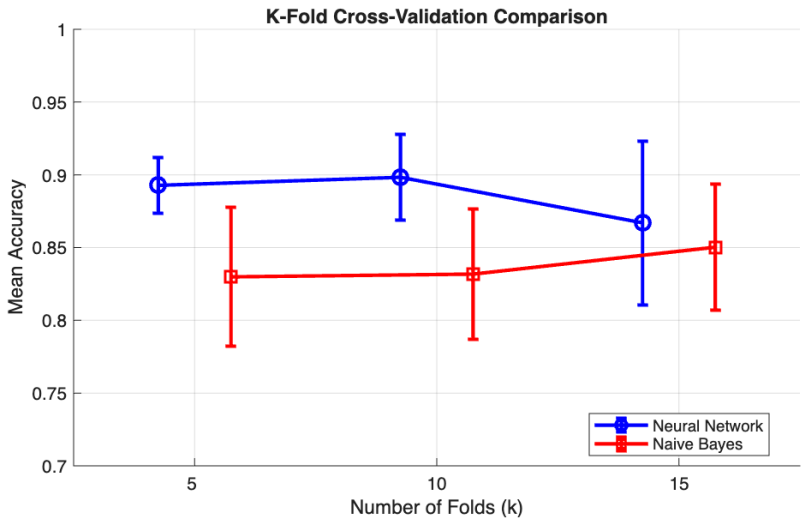

Figure 4: K-fold cross-validation comparison demonstrating $k = 10$ achieves optimal stability-adjusted performance.

Results (Table~2, Figure~4) demonstrated $k = 10$ achieving highest combined stability. Whilst $k = 15$ improved Naive Bayes accuracy marginally, it notably increased neural network variance (5.6% versus 2.9%). The selected $k = 10$ provides ~487 training samples per iteration, balancing sufficient data for parameter estimation against reliable validation.

2.3.4 Neural Network Architecture & Implementation

A pyramidal, feedforward architecture was implemented with progressive feature compression that forces the network to learn increasingly abstract representations: 41 input neurons, two hidden layers (25 and 12 neurons with `patternnet`'s `tansig` activation), and a single sigmoid output neuron. This yields 1,375 trainable parameters: $(41 \times 25 + 25) + (25 \times 12 + 12) + (12 \times 1 + 1)$.

Listing 3: Neural network architecture definition and training

```
1 %Create and configure neural network
```

```
% using patternnet for pattern recognition (classification)
hiddenLayers = [25, 12]; % pyramidal structure; hidden layers narrow toward output
net = patternnet(hiddenLayers, 'trainscg');

% Configure training parameters
net.trainParam.showWindow = false;
net.trainParam.epochs = 200;
net.trainParam.goal = 1e-6;
net.trainParam.min_grad = 1e-7;
net.trainParam.max_fail = 20; % early stopping patience to prevent network from memorisation thus
    ↪ overfitting

% Data division for internal validation (within training set)
% testRatio=0 because I already have an outer-test fold from CV,
% this 80/20 split is for internal early stopping only; final evaluation uses the held-out
    ↪ (never-seen) CV fold
net.divideParam.trainRatio = 0.80;
net.divideParam.valRatio = 0.20;
net.divideParam.testRatio = 0;
```

Hidden layers employ hyperbolic tangent (tanh) activation rather than sigmoid to mitigate vanishing gradients. The sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ peaks at only 0.25; during backpropagation, the chain rule multiplies these derivatives across layers. This causes gradient magnitude to decay significantly for n layers halting learning in early layers (Glorot & Bengio, 2010, pp. 251-252). Tanh, whilst still susceptible to saturation, offers a maximum derivative of 1.0, thereby preserving gradient flow through the network's depth (Glorot & Bengio, 2010, p. 251). The output layer retains sigmoid activation as it directly models the posterior probability $P(\text{PCOS}|\mathbf{x})$, wherein values are naturally bounded to $[0, 1]$.

Scaled conjugate gradient backpropagation (`trainscg`) was employed for computational efficiency, with early stopping (patience=20) preventing overfitting on the limited sample size by observing if validation fails to improve for 20 consecutive epochs. Internal validation used an 80/20 training-validation split within each fold.

2.3.5 Naive Bayes Implementation

The Gaussian distribution was selected based on empirical examination of key clinical features (Figure~5). Hormonal markers (FSH, LH, AMH, TSH) exhibit approximately bell-shaped distributions after z-score normalisation.

$$P(x_i \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} \exp\left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2}\right) \tag{5}$$

wherein parameters $\mu_{k,i}$ (mean) and $\sigma_{k,i}^2$ (variance) are estimated via *Maximum Likelihood Estimation (MLE)* from the training folds. This approach determines the specific-parameter values that maximise the joint probability of observing the training data (Friedman, Geiger and Goldszmidt, 1997, p.136), fitting the optimal bell curve to each feature's class-conditional histogram.

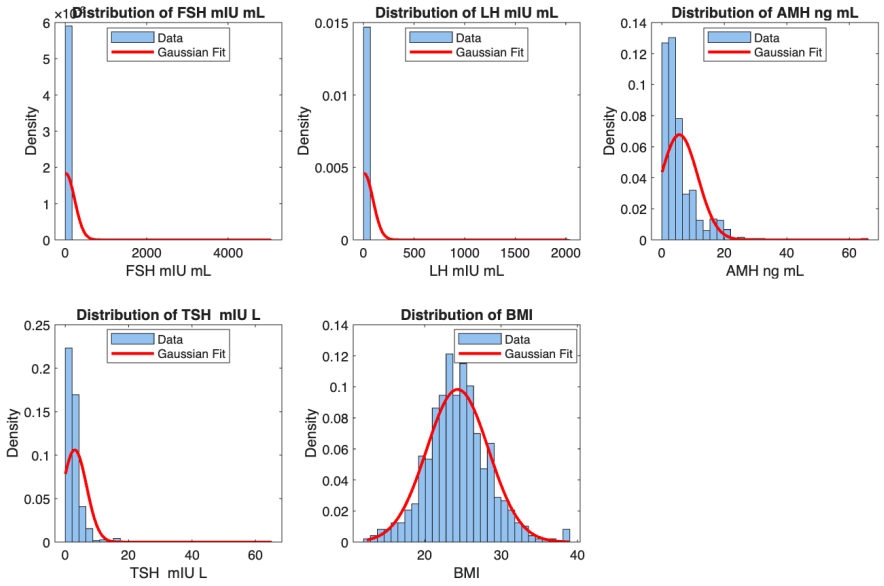

Figure 5: Feature distributions for key clinical biomarkers with Gaussian fits overlaid.

Alternative Distributions Worth Considering: Kernel density estimation works with multimodal distributions but risks overfitting with limited samples per class. Multinomial distribution requires count data i.e. unsuitable for continuous measurements. Bernoulli distribution: a model that restricts to binary features, applicable only to the 8 binary indicators rather than the continuous majority.

Independence assumption: The Gaussian Naive Bayes implementation assumes conditional independence between features given the class label. This assumption is violated in this dataset; for instance, FSH and LH exhibit strong positive correlation, and BMI correlates with weight-related features. However, Zhang (2004, p. 3) states, and thereafter demonstrates, that Naive Bayes remains optimal when dependencies distribute evenly across classes or cancel out across features. The competitive performance observed (84.5% accuracy) suggests balanced dependency effects.

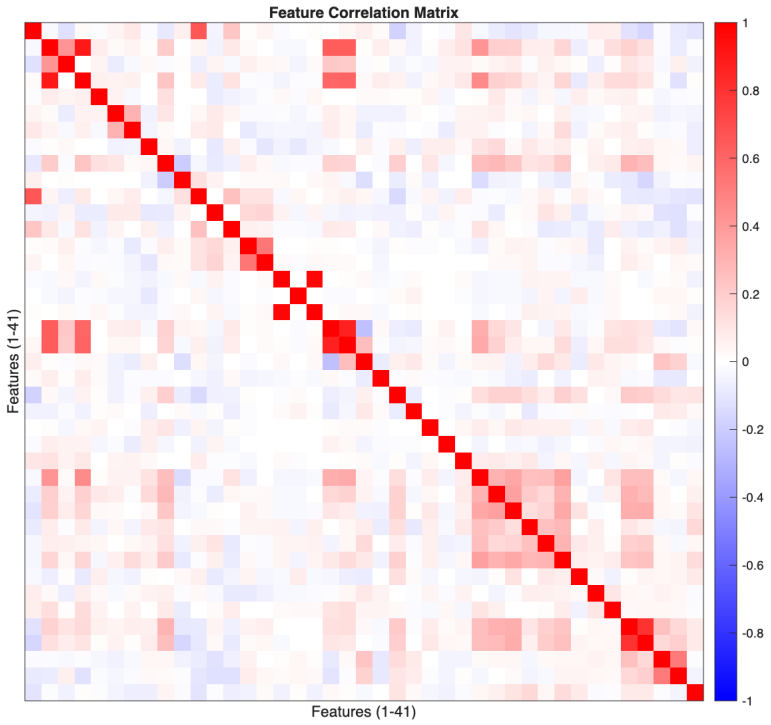

Figure 6: Feature correlation matrix revealing independence assumption violations. Red clusters indicate positively correlated feature groups; off-diagonal (non-zero) structure demonstrates that the Naive Bayes conditional independence assumption $P(\mathbf{x}|C) = \prod_i P(x_i|C)$, wherein the probability of observing all features together given a class equals the product of each feature's individual probability given that class, does not hold. Despite these violations, Naive Bayes achieved 84.5% accuracy, empirically supporting Zhang's (2004, p. 3) theoretical result: classification remains optimal when dependencies distribute symmetrically across classes, preserving posterior rankings even as probability estimates distort.

2.3.6 Results Summary

Table~3 summarises aggregate performance across 10-fold stratified cross-validation.

Table 3: Classification metrics comparison (mean ± std)

Metric	Neural Network	Naive Bayes	Winner
Accuracy	87.8% ± 5.7%	84.5% ± 3.8%	NN
Precision	84.1% ± 10.7%	73.5% ± 7.1%	NN
Recall	78.7% ± 11.0%	84.8% ± 11.1%	NB
F1-Score	80.9% ± 8.9%	78.1% ± 5.1%	NN
Specificity	92.3% ± 5.6%	84.4% ± 6.8%	NN
AUC	0.945 ± 0.032	0.901 ± 0.054	NN
Brier Score	0.088	0.136	NN

The neural network achieves superior discrimination (AUC 0.945 versus 0.901), consistent with its capacity for modelling non-linear feature interactions. However, Naive Bayes demonstrates higher recall (84.8% versus 78.7%), reducing false negatives at the expense of false positives. This performance profile suggests Naive

Bayes may be preferable in screening contexts where the **cost function** penalises missed diagnoses (false negatives) more heavily than false alarms.

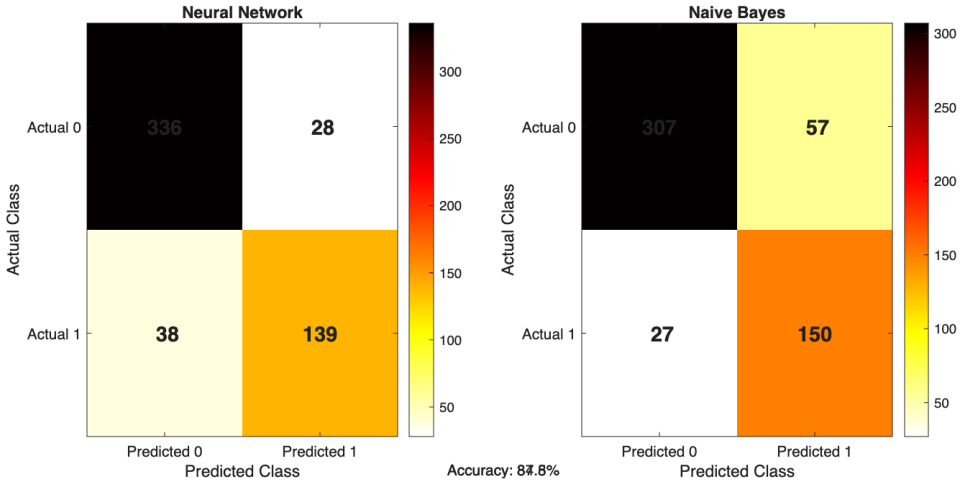

Figure 7: Aggregate confusion matrices across 10-fold cross-validation.

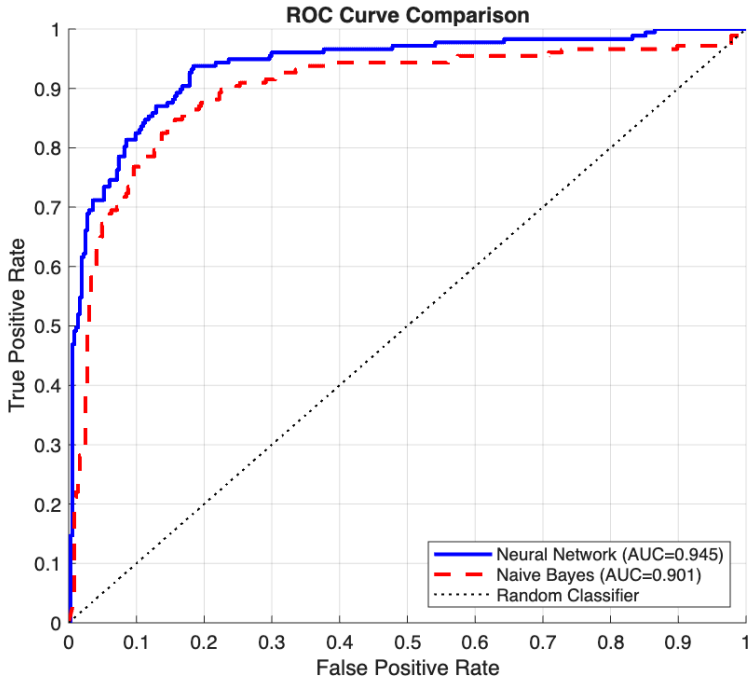

Figure 8: ROC curve comparison demonstrating neural network's superior discrimination.

The Receiver Operating Characteristic (ROC) curves (Figure~8) confirm the neural network's consistent dominance across operating points. Whilst Hand (2009, pp.1-3) highlights the "serious deficiency" of relying solely on the Area Under the Receiver Operating Characteristic Curve (AUC) in medical contexts due to its "incoherent" nature, the observed difference (0.044) remains a primary metric for our initial performance baseline. Naive Bayes completed training in 0.027 seconds versus 0.346 seconds for the neural network,

representing a considerable speedup.

2.4. Discussions and Conclusions (<800*1.1 words)

2.4.1 Benchmarking: Classification Performance

The *NN* achieved 87.8% accuracy, whereas *NB* achieved 84.5% accuracy. Barrera et al. (2023, Abstract) reviewed 31 AI/ML studies for PCOS diagnosis, reporting an accuracy range of 89–100% and AUC spanning 73–100%. Whilst the present 87.8% accuracy falls marginally below the review's lower bound (89%), this discrepancy is likely attributable to methodological rigor; Barrera et al. (2023, p. 4) noted that only 19% of the reviewed studies performed complete *train* → *test* → *validation* pipelines, suggesting many reported higher accuracies via potential overfitting or data leakage. Conversely, the model's 0.945 AUC falls within the review's 73–100% range well, and exceeds the reported median suggesting strong discriminative performance.

Direct comparison with studies using this specific 541-patient Kaggle dataset is informative. Rahman et al. (2023, p. 7) reported 91.01% accuracy using hybrid RFLR, whilst Khanna et al. (2023, Abstract) achieved 98% with multi-stacking. These exceed the present 87.8%, however, neither employed k-fold cross-validation, instead using single train-test splits susceptible to favourable partitioning. The present 10-fold stratified protocol with fold-wise preprocessing prevents such inflation, yielding more generalisable estimates. Kohavi (1995, p. 1137) established 10-fold cross-validation as the "best choice" for datasets under 1000 samples, providing optimal bias-variance trade-off.

The confusion matrices (Figure ~7) reveal a specificity-sensitivity trade-off: the neural network's 92.3% specificity versus 78.7% recall contrasts with Naive Bayes' 84.4% specificity versus 84.8% recall. The neural network favours specificity; Naive Bayes favours recall. Selection depends on which error type the loss function penalises more heavily.

2.4.2 Model Complexity and Interpretability Trade-offs

The performance gap's gains are weighted against models' complexity; the NN requires fitting 1,375 parameters via iterative gradient descent, whilst NB fits only 164 parameters (41 means and 41 variances per class) via closed-form maximum likelihood estimation, training approximately 13× faster (0.027s vs 0.346s across 10 folds).

The NN offers superior discrimination, but it suffers from the "black box" limitation i.e. decision logic distributed across hidden weights ($\mathbf{W}_1 \in \mathbb{R}^{41\times25}$, $\mathbf{W}_2 \in \mathbb{R}^{25\times12}$) when rendering the model opaque to clinicians. Naive Bayes offers transparency in log-odds contributions per feature; the neural network's high parameter count, relative to sample size ($n = 541$, sample-to-parameter ratio of 0.39), presents elevated overfitting risk, mitigated by early stopping; Naive Bayes' low complexity acts as natural regulariser.

2.4.3 Cross-domain Comparisons and the Naive Bayes Paradox

Lestari et al. (2023, p. 8, *of the* `.pdf`) compared neural networks and Naive Bayes on imbalanced medical data, finding NB achieved higher sensitivity (82% versus 78%). This aligns with the present results wherein NB recall (84.8%) outperformed NN's recall (78.7%), suggesting Naive Bayes' independence assumption paradoxically improves sensitivity by preventing overfitting to majority-class patterns.

Zhang (2004, pp. 1-3) proved Naive Bayes remains optimal "no matter how strong the dependences among attributes are… if the dependences distribute evenly in classes, or if the dependences cancel each other out." The correlation matrix (Figure~6) reveals substantive independence assumption violations; red clusters indicate positively correlated feature groups (hormonal markers FSH/LH, anthropometric measures BMI/Weight), whilst the off-diagonal structure confirms features are not conditionally independent given class. Despite this, Naive Bayes achieved 84.5% accuracy — the observed correlation structure suggests dependencies affect PCOS-positive and PCOS-negative cases similarly, preserving posterior rankings (good classifier) even when probability magnitudes distort (bad estimator), thereby explaining competitive performance despite violated assumptions.

2.4.4 Probabilistic Calibration: The Brier Score

The Brier Score quantifies calibration via mean squared error between predicted probabilities and outcomes:

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$$

The neural network achieved 0.088 versus Naive Bayes' 0.136, a 35% relative improvement. As a Brier score of 0 represents a perfect model and 0.25 represents a non-informative baseline (Steyerberg et al., 2010, p. 129), both models demonstrate acceptable calibration, although the neural network's lower score reflects superior probability estimates that clinicians can trust more reliably. Naive Bayes' weaker calibration stems from treating correlated markers as independent, effectively "double-counting" evidence and pushing posteriors toward extremes. This aligns with Zhang's (2004, p. 3) analysis: Naive Bayes achieves optimal *classification* despite producing miscalibrated *probability estimates*.

2.4.5 Methodology and Limitations

Choosing 10-fold stratified cross-validation aligns well with Kohavi's (1995, p. 6, Summary) recommendation for datasets under 1000 samples. However, Barrera et al. (2023, p. 4) noted only 19% of PCOS studies employed proper validation; the present fold-wise preprocessing prevents data leakage that inflates metrics, explaining the seemingly lower accuracy compared to studies reporting 95%+ without cross-validation.

The k-fold comparison (Figure~4) reveals neural network accuracy variance increases at $k = 15$ whilst Naive Bayes remains stable across folds. This supports the $k = 10$ selection; larger k values reduce training set size per fold, disproportionately affecting the parameter-heavy neural network whilst leaving *NB's* closed-form estimates relatively unaffected.

Limitations: The 2.06:1 class imbalance likely biased both models towards the majority class (Lestari et al. demonstrated SMOTE improved NB sensitivity from 82% to 89%); the Kaggle dataset lacks provenance verification, as only 32% of PCOS machine learning studies employed standardised diagnostic criteria (Barrera et al., 2023, p. 3).

2.4.6 Conclusion

Classifier choice depends on the application's loss function and interpretability requirements. The neural network's superior discrimination (0.945 AUC, exceeding Panjwani et al.'s 2025 optimised ensemble, pp. 25-26) suits contexts prioritising predictive power over transparency. Conversely, Naive Bayes' higher recall (84.8% versus 78.7%) and explicit log-odds contributions suit contexts requiring model explainability or where the cost function penalises false negatives more heavily.

References (Dataset and Peer-reviewed Papers)

- *Data set:* Kottarathil, P. (2020) Polycystic Ovary Syndrome (PCOS). Kaggle. Available at: https://www.kaggle.com/datasets/prasoonkottarathil/polycystic-ovary-syndrome-pcos?resource=download (Accessed: 3rd December 2025)

Task (3)'s

- Bellman, R. (1957) 'A Markovian Decision Process', Journal of Mathematics and Mechanics, 6(5), pp. 679-684. Available at: https://www.jstor.org/stable/24900506 (Accessed: 14 November 2025).
- Even-Dar, E. and Mansour, Y. (2003) 'Learning rates for Q-learning', Journal of Machine Learning Research, 5(Dec), pp. 1 to 25. Available at: https://www.jmlr.org/papers/volume5/evendar03a/evendar03a.pdf (Accessed: 5 December 2025).

- Kaelbling, L.P., Littman, M.L. and Moore, A.W. (1996) 'Reinforcement learning: A survey', Journal of Artificial Intelligence Research, 4, pp. 237 to 285. Available at: https://www.jair.org/index.php/jair/article/view/10166/24110 (Accessed: 20 November 2025).
- Mnih, V. et al. (2015) 'Human-level control through deep reinforcement learning', Nature, 518(7540), pp. 529-533. Available at: https://web.stanford.edu/class/psych209/Readings/MnihEtAlHassibis15NatureControlDeepRL.pdf (Accessed: 10 December 2025).
- Tsitsiklis, J.N. (1994) 'Asynchronous stochastic approximation and Q-learning', Machine Learning, 16(3), pp. 185-202. Available at: https://www.mit.edu/~jnt/Papers/J052-94-jnt-q.pdf (Accessed: 11 December 2025).
- Watkins, C.J.C.H. and Dayan, P. (1992) 'Q-learning', Machine Learning, 8(3-4), pp. 279 to 292. Available at: https://link.springer.com/article/10.1007/BF00992698 (Accessed: 5 December 2025).

Task (4)'s

- Domingos, P. and Pazzani, M. (1997) 'On the optimality of the simple Bayesian classifier under zero-one loss', Machine Learning, 29(2-3), pp. 103-130. Available at: https://link.springer.com/article/10.1023/A:1007413511361 (Accessed: 10 December 2025).
- Kotsiantis, S.B. (2007) 'Supervised machine learning: A review of classification techniques', Informatica, 31(3), pp. 249-268. Available at: https://datajobs.com/data-science-repo/Supervised-Learning-%5BSB-Kotsiantis%5D.pdf (Accessed: 12 December 2025).
- Smith, L.N. (2017) 'Cyclical learning rates for training neural networks', in 2017 IEEE Winter Conference on Applications of Computer Vision (WACV). IEEE, pp. 464-472. Available at: https://sands.kaust.edu.sa/classes/CS290E/F19/papers/clr.pdf (Accessed: 20 December 2025).
- Bergstra, J. and Bengio, Y. (2012) 'Random search for hyper-parameter optimization', Journal of Machine Learning Research, 13, pp. 281-305. Available at: https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf (Accessed: 21 December 2025).
- Friedman, N., Geiger, D. and Goldszmidt, M. (1997) 'Bayesian network classifiers', Machine Learning, 29(2-3), pp. 131-163. Available at: https://link.springer.com/article/10.1023/A:1007465528199 (Accessed: 22 December 2025).
- Hornik, K., Stinchcombe, M. and White, H. (1989) 'Multilayer feedforward networks are universal approximators', Neural Networks, 2(5), pp. 359-366. Available at: https://www.sciencedirect.com/science/article/abs/pii/0893608089900208 (Accessed: 23 December 2025).
- Manning, C.D., Raghavan, P. and Schütze, H. (2008) Introduction to Information Retrieval. Cambridge: Cambridge University Press. Available at: https://nlp.stanford.edu/IR-book/pdf/irbookonlinereading.pdf (Accessed: 24 December 2025).
- Lizneva, D., Suturina, L., Walker, W., Brakta, S., Gavrilova-Jordan, L. and Azziz, R. (2016) 'Criteria, prevalence, and phenotypes of polycystic ovary syndrome', Fertility and Sterility, 106(1), pp. 6-15. Available at: https://www.fertstert.org/article/S0015-0282(16)61232-3/fulltext (Accessed: 24 December 2025).
- Zhang, H. (2004) 'The optimality of naive Bayes', in FLAIRS Conference. Available at: https://www.cs.unb.ca/~hzhang/publications/FLAIRS04ZhangH.pdf (Accessed: 26 December 2025).
- Glorot, X. and Bengio, Y. (2010) 'Understanding the difficulty of training deep feedforward neural networks', in Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, pp. 249-256. Available at: https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf (Accessed: 27 December 2025).
- Hand, D.J. (2009) 'Measuring classifier performance: a coherent alternative to the area under the ROC curve', Machine Learning, 77(1), pp. 103-123. Available at: https://link.springer.com/article/10.1007/s10994-009-5119-5 (Accessed: 27 December 2025).
- Amato, F., López, A., Peña-Méndez, E. M., Vaňhara, P., Hampl, A., & Havel, J. (2013). 'Artificial neural networks in medical diagnosis'. Journal of Applied Biomedicine, 11(2), 47-58. https://doi.org/10.2478/v10136-012-0031-x (Accessed: 29 December 2025).
- Barrera, F.J. et al. (2023) 'Application of machine learning and artificial intelligence in the diagnosis and classification of polycystic ovarian syndrome: a systematic review', Frontiers in Endocrinology, 14, p. 1106625. Available at: https://doi.org/10.3389/fendo.2023.1106625 (Accessed: 28 December 2025).

- Panjwani, B., Yadav, J., Mohan, V., Agarwal, N. and Agarwal, S. (2025). 'Optimized Machine Learning for the Early Detection of Polycystic Ovary Syndrome in Women'. Sensors, 25(4), 1166. Available at: https://doi.org/10.3390/s25041166. (Accessed: 30 December 2025).
- Rahman , M.M., Islam, M.M., Islam, M.R. and Al-Mamun, M.R. (2023). 'A Hybrid Machine Learning Approach for Early Detection of Polycystic Ovary Syndrome.' Computational Intelligence and Neuroscience, 2023. Available at: https://doi.org/10.1155/2023/9925893. (Accessed: 29 December 2025).
- Khanna, V. V., Chadaga, K., Sampathila, N., Prabhu, S., Bhandage, V., & Hegde, G. K. (2023). 'A Distinctive Explainable Machine Learning Framework for Detection of Polycystic Ovary Syndrome'. Applied System Innovation, 6(2), 32. https://doi.org/10.3390/asi6020032 (Accessed: 29 December 2025).
- Kohavi, R. (1995). 'A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection'. In Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI), Vol. 2, pp. 1137-1143. Available at : https://ai.stanford.edu/~ronnyk/accEst.pdf (Accessed: 30 December 2025).
- Lestari, N., Indahwati, Erfiani and Julianti, E.D. (2023) 'A Comparison of Artificial Neural Network and Naïve Bayes Classification Using Unbalanced Data Handling', BAREKENG: Journal of Mathematics and Its Applications, 17(3), pp. 1585-1594. Available at: https://www.researchgate.net/publication/374346099_A_COMPARISON_OF_ARTIFICIAL_NEURAL_NETWORK_AND_NAIVE_BAYES_CLASSIFICATION_USING_UNBALANCED_DATA_HANDLING(Accessed: 31 December 2025).
- Steyerberg, E. W., Vickers, A. J., Cook, N. R., Gerds, T., Gonen, M., Obuchowski, N., Pencina, M. J., & Kattan, M. W. (2010). 'Assessing the performance of prediction models: a framework for traditional and novel measures.' Epidemiology, 21(1), 128-138. https://www.researchgate.net/publication/40687783_Assessing_the_Performance_of_Prediction_Models_A_Framework_for_Traditional_and_Novel_Measures (Accessed: 31 December 2025).

Appendix

Appendix A: MATLAB Code

```
%% COMP3003 Report; Task (4) Classification
% Binary classification: Neural Network (NN) vs Naive Bayes (NB) on Polycystic Ovary Syndrome (PCOS) clinical data
% Dataset: PCOS Full Dataset (Kaggle) - 541 patients; 41 predictive features

%{
- [X] (a) load and clean dataset
- [X] (b) implement fold-wise preprocessing (imputation + normalisation from train only)
- [X] (c) test k = 5, 10, 15 and justify selection experimentally
- [X] (d) build pyramidal-structured NN [25, 12]
- [X] (e) build Gaussian NB with distribution justification
- [X] (f) compute metrics: accuracy, precision, recall, F1, specificity, AUC
%}
clear; clc; close all;
rng(42); % constant seed for consistent results to observe changes


%% (a) Data Loading and Exploration %%
fprintf('COMP 3003 Task (4): Classification\n\n');
fprintf('(a) Loading and Exploring the Dataset...\n');

%Load the PCOS dataset
rawData = readtable('PCOS_full_dataset.csv', 'VariableNamingRule', 'preserve');
[numRows, numCols] = size(rawData);
fprintf('Initial dimensions: %d rows x %d columns\n', numRows, numCols);
```

```
25
26 % Display original column names for verification
27 fprintf('\nOriginal column names:\n');
28 for i = 1:numCols
29     fprintf(' Col %d: "%s"\n', i, rawData.Properties.VariableNames{i});
30 end
31
32 colsToRemove = {'Sl. No', 'Patient File No.', 'Unnamed: 44'}; % unimportant predictive values;
       ↪ non-predictive identifiers
33 fprintf('\nRemoving non-predictive columns:\n');
34 for i = 1:length(colsToRemove) % for each non-predictive column, if it exists, remove it to
       ↪ prevent memorisation, i.e. when the model learns to associate specific IDs with labels seen
       ↪ during training runs thereby overfitting
35     colName = colsToRemove{i};
36     if any(strcmp(rawData.Properties.VariableNames, colName))
37         rawData(:, colName) = [];
38         fprintf(' - Removed: "%s"\n', colName);
39     end
40 end
41
42 % Extract target variable prior to modifying remaining columns
43 % 0 = NOT-PCOS; 1 = PCOS
44 targetColName = 'PCOS (Y/N)';
45 labels = rawData.(targetColName);
46 rawData(:, targetColName) = [];
47
48 fprintf('\nAfter cleaning: %d samples x %d features\n', height(rawData), width(rawData));
49
50 % Clean the column names (strip whitespace, standardise naming)
51 cleanedNames = rawData.Properties.VariableNames;
52 for i = 1:length(cleanedNames)
53     % Remove leading/trailing whitespace
54     cleanedNames{i} = strtrim(cleanedNames{i});
55     % Replace certain problematic characters for MATLAB compliance
56     cleanedNames{i} = strrep(cleanedNames{i}, ' I ', 'I_');
57     cleanedNames{i} = strrep(cleanedNames{i}, 'II ', 'II_');
58     cleanedNames{i} = strrep(cleanedNames{i}, ' ', '_');
59     cleanedNames{i} = strrep(cleanedNames{i}, '(', '_');
60     cleanedNames{i} = strrep(cleanedNames{i}, ')', '');
61     cleanedNames{i} = strrep(cleanedNames{i}, '.', '');
62     cleanedNames{i} = strrep(cleanedNames{i}, ':', '_');
63     cleanedNames{i} = strrep(cleanedNames{i}, '/', '_');
64 end
65 rawData.Properties.VariableNames = cleanedNames;
66
67 %{
68     Fix; handled known dirty columns containing text typos (e.g. "II")
69
70     This raw CSV contains cells with "II" instead of numeric values, causing
71     MATLAB to import entire columns as text/cell arrays instead of doubles.
72     This must be fixed before table2array otherwise numeric operations fail due
73     to type mismatch.
74 %}
75 fprintf('\nChecking for dirty columns with text values...\n');
76 for i = 1:width(rawData)
77     colName = rawData.Properties.VariableNames{i};
78     colData = rawData.(colName);
79
80     %Check if column is cell or string (indicates non-numeric data)
```

```
    if iscell(colData) || isstring(colData)
        fprintf(' - Fixing dirty column: %s (converting text to numeric)\n', colName);
        % force conversion: text like "II" becomes NaN, thus only valid numbers remain
        rawData.(colName) = str2double(colData);
    end
end

% Extract numeric data and dimensions
data = table2array(rawData); % convert cleaned table into a numerical matrix for processing
featureNames = rawData.Properties.VariableNames;
numFeatures = size(data, 2);
numSamples = size(data, 1);

fprintf('\nFinal dataset: %d samples x %d features\n', numSamples, numFeatures);

% Check for missing values
missingCount = sum(isnan(data(:))); % true if not-a-number; sum counts total missing entries
fprintf('Missing values detected: %d\n', missingCount);

% Missing values left as NaN; imputation (filling in with estimates) done within each fold
% to prevent data leakage (test data must not influence preprocessing)
if missingCount > 0
    fprintf('Missing values will be imputed within each CV fold (leakage prevention)\n');
end

% Class distribution analysis
numPCOS = sum(labels == 1);
numNonPCOS = sum(labels == 0);
fprintf('\nClass Distribution:\n');
fprintf(' - PCOS positive (1): %d (%.1f%%)\n', numPCOS, 100*numPCOS/numSamples); % calculate
    ↪ percentage of positive class
fprintf(' - PCOS negative (0): %d (%.1f%%)\n', numNonPCOS, 100*numNonPCOS/numSamples);
imbalanceRatio = max(numPCOS, numNonPCOS) / min(numPCOS, numNonPCOS);
fprintf(' - Imbalance ratio: %.2f:1\n', imbalanceRatio);

% Plot class distribution
figure('Name', 'Class Distribution', 'Position', [100, 100, 500, 400]);
bar([numNonPCOS, numPCOS], 'FaceColor', [0.3, 0.6, 0.9]);
set(gca, 'XTickLabel', {'Non-PCOS (0)', 'PCOS (1)'});
ylabel('Number of Samples');
title('Class Distribution in PCOS Dataset');
text(1, numNonPCOS + 10, sprintf('%d (%.1f%%)', numNonPCOS, 100*numNonPCOS/numSamples),
    ↪ 'HorizontalAlignment', 'center');
text(2, numPCOS + 10, sprintf('%d (%.1f%%)', numPCOS, 100*numPCOS/numSamples),
    ↪ 'HorizontalAlignment', 'center');
grid on;
saveas(gcf, 'fig_classDistribution.png');
fprintf('\nSaved: fig_classDistribution.png\n');


%% (b) Preprocessing and Normalisation %%
fprintf('\n(b) Preprocessing and Normalisation...\n');

%Identify feature types based on unique value counts and domain knowledge,
% binary features; 0/1 values only thus no normalisation needed:
binaryFeatureNames = {
    'Pregnant_Y_N'
    'Weight_gain_Y_N'
    'hair_growth_Y_N'
```

```
    'Skin_darkening__Y_N'
    'Hair_loss_Y_N'
    'Pimples_Y_N'
    'Fast_food__Y_N'
    'RegExercise_Y_N'
};

% Categorical features; encoded as integers
categoricalFeatureNames = {'Blood_Group', 'Cycle_R_I'};

% Find indices of binary and categorical features
binaryIdx = false(1, numFeatures);
categoricalIdx = false(1, numFeatures);
for i = 1:numFeatures
    % Check if binary feature contains 'Y_N' in name
    if contains(featureNames{i}, 'Y_N')
        binaryIdx(i) = true;
    end
    % Check for categorical features
    if any(strcmpi(featureNames{i}, categoricalFeatureNames))
        categoricalIdx(i) = true;
    end
end

% Continuous features, i.e., non-binary and non-categorical
continuousIdx = ~binaryIdx & ~categoricalIdx; % if not binary or categorical, then is continuous

fprintf('\nFeature type breakdown:\n');
fprintf(' - Continuous features: %d (to be normalised)\n', sum(continuousIdx));
fprintf(' - Binary features: %d (left as 0/1)\n', sum(binaryIdx));
fprintf(' - Categorical features: %d (left as numeric codes)\n', sum(categoricalIdx));

% Check for outliers in continuous features
fprintf('\nDetecting outliers in continuous features:\n');
continuousData = data(:, continuousIdx);
continuousNames = featureNames(continuousIdx);
for i = 1:size(continuousData, 2)
    colData = continuousData(:, i);
    colData = colData(~isnan(colData)); % remove NaNs before percentile calc
    if isempty(colData)
        continue;
    end
    q1 = prctile(colData, 25);
    q3 = prctile(colData, 75);
    iqr = q3 - q1;
    lowerBound = q1 - 3*iqr; % using 3*IQR for extreme outliers only (Interquartile Range method,
        ↪ i.e. stops flagging mild thus chance-outliers)
    upperBound = q3 + 3*iqr;
    outliers = colData < lowerBound | colData > upperBound;
    if any(outliers)
        fprintf(' - %s: %d extreme outliers detected\n', ...
            continuousNames{i}, sum(outliers));
    end
end


%% (c) Cross-Validation Setup and K-Selection %%
fprintf('\n(c) Cross-Validation Setup and K-Selection...\n');

```

```
% Test k = 5, 10, 15 folds to determine optimal value
% For 541 samples:
%   k=5 -> 108 samples per fold
%   k=10 -> 54 samples per fold
%   k=15 -> 36 samples per fold

kValues = [5, 10, 15];

fprintf('\nTesting different k-fold values...\n');
fprintf('Dataset size: %d samples\n', numSamples);
for k = kValues
    foldSize = floor(numSamples / k);
    fprintf(' k=%d -> ~%d samples per fold\n', k, foldSize);
end

% Run quick NN and NB evaluation for each k to determine optimal
fprintf('\nRunning k-fold comparison experiments...\n');
kAccuraciesNN = zeros(length(kValues), 1);
kAccuraciesNB = zeros(length(kValues), 1);
kStdNN = zeros(length(kValues), 1);
kStdNB = zeros(length(kValues), 1);

for kidx = 1:length(kValues) % loop over k values
    k = kValues(kidx);
    fprintf('\n--- Testing k=%d ---\n', k);

    % Create stratified k-fold partition to preserve class ratios; without it some inconsistent
        ↪ folds may skew results due to under-representation of minority class
    cv = cvpartition(labels, 'KFold', k, 'Stratify', true);

    foldAccNN = zeros(k, 1);
    foldAccNB = zeros(k, 1);

    for fold = 1:k
        % Get training and test indices
        trainIdx = training(cv, fold);
        testIdx = test(cv, fold);

        % Extract raw data to prevent leakage
        xTrainRaw = data(trainIdx, :);
        yTrain = labels(trainIdx);
        xTestRaw = data(testIdx, :);
        yTest = labels(testIdx);

        % Fold-wise preprocessing (imputation + normalisation from train only)

        % Important: Impute test set using train means, not test means,
        % if test data influenced imputation, model would have indirect knowledge of test
            ↪ distribution (data leakage)
        trainMeans = mean(xTrainRaw, 'omitnan'); % omit NaNs when calculating means
        xTrain = fillmissing(xTrainRaw, 'constant', trainMeans);
        xTest = fillmissing(xTestRaw, 'constant', trainMeans);

        mu = mean(xTrain, 1);
        sigma = std(xTrain, 0, 1);
        sigma(sigma == 0) = 1; % prevent division by zero for constant features
        xTrain = (xTrain - mu) ./ sigma; % element-wise normalisation of training set
        xTest = (xTest - mu) ./ sigma;

```

```
        % Quick Neural Network (simplified for k-selection)
        net = patternnet(20, 'trainscg'); % Scaled Conjugate Gradient (`trainscg`): avoid computational cost of line searches
        net.trainParam.showWindow = false;
        net.trainParam.epochs = 100;
        net.divideParam.trainRatio = 0.85;
        net.divideParam.valRatio = 0.15;
        net.divideParam.testRatio = 0;

        net = train(net, xTrain', yTrain');
        nnPred = net(xTest');
        nnPredClass = round(nnPred)';
        foldAccNN(fold) = sum(nnPredClass == yTest) / length(yTest);

        % Naive Bayes (Gaussian distribution for continuous features)
        nbModel = fitcnb(xTrain, yTrain, 'DistributionNames', 'normal');
        nbPred = predict(nbModel, xTest);
        foldAccNB(fold) = sum(nbPred == yTest) / length(yTest);
    end

    kAccuraciesNN(kidx) = mean(foldAccNN);
    kAccuraciesNB(kidx) = mean(foldAccNB);
    kStdNN(kidx) = std(foldAccNN);
    kStdNB(kidx) = std(foldAccNB);

    fprintf(' NN: %.3f +/- %.3f\n', kAccuraciesNN(kidx), kStdNN(kidx));
    fprintf(' NB: %.3f +/- %.3f\n', kAccuraciesNB(kidx), kStdNB(kidx));
end

% Display k-fold comparison table
fprintf('\n--- K-Fold Comparison Results ---\n');
fprintf('%-5s | %-20s | %-20s\n', 'k', 'Neural Network', 'Naive Bayes');
fprintf('------+----------------------+----------------------\n');
for kidx = 1:length(kValues)
    fprintf('%-5d | %.3f +/- %.3f | %.3f +/- %.3f\n', ...
        kValues(kidx), kAccuraciesNN(kidx), kStdNN(kidx), ...
        kAccuraciesNB(kidx), kStdNB(kidx));
end

% Select optimal k based on experimental results
% Using stability-adjusted performance (penalise high variance)
stabilityScoreNN = kAccuraciesNN - 0.5*kStdNN; % this will penalise high-variance k values to prefer stable estimates
stabilityScoreNB = kAccuraciesNB - 0.5*kStdNB;
combinedScore = (stabilityScoreNN + stabilityScoreNB) / 2;

[~, bestKIdx] = max(combinedScore);
optimalK = kValues(bestKIdx);

fprintf('\nSelected k=%d based on stability-adjusted performance\n', optimalK);
fprintf('Justification:\n');
fprintf(' - Highest combined stability score across both models\n');
fprintf(' - Balances accuracy with low variance for reliable estimates\n');

% Plot k-fold comparison
figure('Name', 'K-Fold Comparison', 'Position', [100, 100, 700, 400]);
x = 1:length(kValues);
hold on;
errorbar(x - 0.15, kAccuraciesNN, kStdNN, 'b-o', 'LineWidth', 2, 'MarkerSize', 8); % NN with
```

```
    ↪ slight left offset
errorbar(x + 0.15, kAccuraciesNB, kStdNB, 'r-s', 'LineWidth', 2, 'MarkerSize', 8); % NB with
    ↪ slight right offset
hold off;
set(gca, 'XTick', x, 'XTickLabel', kValues);
xlabel('Number of Folds (k)');
ylabel('Mean Accuracy');
title('K-Fold Cross-Validation Comparison');
legend('Neural Network', 'Naive Bayes', 'Location', 'best');
grid on;
ylim([0.7, 1.0]);
saveas(gcf, 'fig_kfoldComparison.png');
fprintf('Saved: fig_kfoldComparison.png\n');

%% (d) Neural Network Implementation %%
fprintf('\n(d) Neural Network Implementation...\n');

% Design:
%   - Input: 41 features
%   - Hidden layer 1: 25 neurons (between input and output, ~60% of input)
%   - Hidden layer 2: 12 neurons (pyramidal structure for abstraction)
%   - Output: 1 neuron (sigmoid for binary classification)

fprintf('\nNeural Network Architecture:\n');
fprintf(' - Input layer: %d neurons (one per feature)\n', numFeatures);
fprintf(' - Hidden layer 1: 25 neurons (tanh activation)\n');
fprintf(' - Hidden layer 2: 12 neurons (tanh activation)\n');
fprintf(' - Output layer: 1 neuron (sigmoid activation)\n');

% Create stratified k-fold partition for final evaluation
cv = cvpartition(labels, 'KFold', optimalK, 'Stratify', true);

% Storage for NN results; initialise metrics
nnFoldMetrics = struct();
nnFoldMetrics.accuracy = zeros(optimalK, 1); % Overall Accuracy: (TP + TN) / (TP + TN + FP + FN)
nnFoldMetrics.precision = zeros(optimalK, 1); % Positive Predictive Value: TP / (TP + FP)
nnFoldMetrics.recall = zeros(optimalK, 1); % Sensitivity: TP / (TP + FN)
nnFoldMetrics.f1 = zeros(optimalK, 1); % F1 Score: harmonic mean of precision and recall
nnFoldMetrics.specificity = zeros(optimalK, 1); % True Negative Rate: TN / (TN + FP)
nnFoldMetrics.auc = zeros(optimalK, 1); % Area Under the Curve for ROC: probability ranking quality

nnTrainTimes = zeros(optimalK, 1); % store time-taken for each fold's training

% Store predictions for final confusion matrix and ROC
allNNPred = [];
allNNScores = [];
allNNTrue = [];

fprintf('\nTraining Neural Network across %d folds...\n', optimalK);

for fold = 1:optimalK
    fprintf(' Fold %d/%d: ', fold, optimalK);

    trainIdx = training(cv, fold);
    testIdx = test(cv, fold);

    % Extract raw data (before normalisation) to prevent leakage
    xTrainRaw = data(trainIdx, :);
    yTrain = labels(trainIdx);
```

```
    xTestRaw = data(testIdx, :);
    yTest = labels(testIdx);

    %Fold-wise preprocessing; prevents data leakage
    % 1- impute missing values using TRAINING means only
    trainMeans = mean(xTrainRaw, 'omitnan');
    xTrain = fillmissing(xTrainRaw, 'constant', trainMeans);
    xTest = fillmissing(xTestRaw, 'constant', trainMeans);

    % 2- Z-score normalise using training statistics only
    mu = mean(xTrain, 1);
    sigma = std(xTrain, 0, 1);
    sigma(sigma == 0) = 1; % setting sigma=1 leaves constant features unchanged to avoid division
        ↪ by zero thus NaN

    xTrain = (xTrain - mu) ./ sigma;
    xTest = (xTest - mu) ./ sigma; % divide element-wise

    % Create and configure neural network
    % using patternnet for pattern recognition (classification)
    hiddenLayers = [25, 12]; % pyramidal structure; hidden layers narrow toward output
    net = patternnet(hiddenLayers, 'trainscg');

    % Configure training parameters
    net.trainParam.showWindow = false;
    net.trainParam.epochs = 200;
    net.trainParam.goal = 1e-6;
    net.trainParam.min_grad = 1e-7;
    net.trainParam.max_fail = 20; % early stopping patience to prevent network from memorisation
        ↪ thus overfitting

    % Data division for internal validation (within training set)
    % testRatio=0 because I already have an outer-test fold from CV,
    % this 80/20 split is for internal early stopping only; final evaluation uses the held-out CV
        ↪ fold
    net.divideParam.trainRatio = 0.80;
    net.divideParam.valRatio = 0.20;
    net.divideParam.testRatio = 0;

    % Train the network
    tic; % start timer
    net = train(net, xTrain', yTrain');
    nnTrainTimes(fold) = toc; % stop timer and record duration

    % Get predictions and scores
    nnScores = net(xTest')';
    fprintf(' Score range: [%.4f, %.4f]\n', min(nnScores), max(nnScores));
    fprintf(' Unique predictions: %d\n', length(unique(round(nnScores))));
    nnPred = double(nnScores >= 0.5); % 0.5 ensures symmetric mis-classification costs; can tune
        ↪ threshold later if recall matters more

    % Store for aggregate analysis
    allNNPred = [allNNPred; nnPred];
    allNNScores = [allNNScores; nnScores];
    allNNTrue = [allNNTrue; yTest];

    % Compute fold metrics
    [acc, prec, rec, f1, spec] = ComputeMetrics(yTest, nnPred);

```

```
    % Compute AUC; the tildes (~) ignore the X,Y coordinates of the ROC curve as only the scalar AUC is relevant, the 1 defines the positive class label, 0=negative class
    [~, ~, ~, auc] = perfcurve(yTest, nnScores, 1);

    nnFoldMetrics.accuracy(fold) = acc;
    nnFoldMetrics.precision(fold) = prec;
    nnFoldMetrics.recall(fold) = rec;
    nnFoldMetrics.f1(fold) = f1;
    nnFoldMetrics.specificity(fold) = spec;
    nnFoldMetrics.auc(fold) = auc;

    fprintf('Acc=%.3f, F1=%.3f, AUC=%.3f\n', acc, f1, auc);
end

% Compute aggregate NN metrics
fprintf('\n--- Neural Network Results (Mean +/- Std) ---\n');
fprintf(' Accuracy:  %.3f +/- %.3f\n', mean(nnFoldMetrics.accuracy), std(nnFoldMetrics.accuracy));
fprintf(' Precision: %.3f +/- %.3f\n', mean(nnFoldMetrics.precision), std(nnFoldMetrics.precision));
fprintf(' Recall:    %.3f +/- %.3f\n', mean(nnFoldMetrics.recall), std(nnFoldMetrics.recall));
fprintf(' F1-Score:  %.3f +/- %.3f\n', mean(nnFoldMetrics.f1), std(nnFoldMetrics.f1));
fprintf(' Specificity: %.3f +/- %.3f\n', mean(nnFoldMetrics.specificity), std(nnFoldMetrics.specificity));
fprintf(' AUC:       %.3f +/- %.3f\n', mean(nnFoldMetrics.auc), std(nnFoldMetrics.auc));

% Plot NN confusion matrix (aggregate)
figure('Name', 'NN Confusion Matrix', 'Position', [100, 100, 500, 450]);
PlotConfusionMatrix(allNNTrue, allNNPred, 'Neural Network Confusion Matrix');
saveas(gcf, 'fig_confusionNN.png');
fprintf('\nSaved: fig_confusionNN.png\n');


%% (e) Naive Bayes Implementation %%

fprintf('\n(e) Naive Bayes Implementation...\n');

% First plot feature distributions to justify distribution choice
fprintf('\nPlotting feature distributions for distribution selection...\n');

% Select key clinical features for distribution analysis
keyFeatures = {'FSH_mIU_mL', 'LH_mIU_mL', 'AMH_ng_mL', 'TSH__mIU_L', 'BMI'};
keyFeatureIdx = zeros(1, length(keyFeatures));
for i = 1:length(keyFeatures)
    idx = find(contains(featureNames, keyFeatures{i}), 1);
    if ~isempty(idx)
        keyFeatureIdx(i) = idx;
    end
end
keyFeatureIdx = keyFeatureIdx(keyFeatureIdx > 0);

figure('Name', 'Feature Distributions', 'Position', [100, 100, 1000, 600]);
for i = 1:min(6, length(keyFeatureIdx))
    subplot(2, 3, i);
    idx = keyFeatureIdx(i);
    histogram(data(:, idx), 30, 'Normalization', 'pdf', 'FaceColor', [0.3, 0.6, 0.9]);
    xlabel(strrep(featureNames{idx}, '_', ' '));
    ylabel('Density');
    title(sprintf('Distribution of %s', strrep(featureNames{idx}, '_', ' ')));

```

```
    % Overlay normal distribution fit onto histogram
    hold on;
    col = data(:, idx);
    col = col(~isnan(col));

    x = linspace(min(col), max(col), 100); % 1- range for PDF
    mu = mean(col); % 2- calculated mean of column
    sigma = std(col); % 3- calculated standard deviation of column
    y = normpdf(x, mu, sigma);
    plot(x, y, 'r-', 'LineWidth', 2);
    hold off;
    legend('Data', 'Gaussian Fit', 'Location', 'best');
end
saveas(gcf, 'fig_featureDistributions.png');
fprintf('Saved: fig_featureDistributions.png\n');

% Train Naive Bayes with same k-fold CV as NN
nbFoldMetrics = struct();
nbFoldMetrics.accuracy = zeros(optimalK, 1);
nbFoldMetrics.precision = zeros(optimalK, 1);
nbFoldMetrics.recall = zeros(optimalK, 1);
nbFoldMetrics.f1 = zeros(optimalK, 1);
nbFoldMetrics.specificity = zeros(optimalK, 1);
nbFoldMetrics.auc = zeros(optimalK, 1);

nbTrainTimes = zeros(optimalK, 1); % store time-taken duration

allNBPred = [];
allNBScores = [];
allNBTrue = [];

fprintf('\nTraining Naive Bayes across %d folds...\n', optimalK);

for fold = 1:optimalK
    fprintf(' Fold %d/%d: ', fold, optimalK);

    trainIdx = training(cv, fold);
    testIdx = test(cv, fold);

    % Extract raw data before normalisation to prevent leakage
    xTrainRaw = data(trainIdx, :);
    yTrain = labels(trainIdx);
    xTestRaw = data(testIdx, :);
    yTest = labels(testIdx);

    % Fold-wise preprocessing to prevent data leakage
    % 1- Impute missing values using training means only
    trainMeans = mean(xTrainRaw, 'omitnan');
    xTrain = fillmissing(xTrainRaw, 'constant', trainMeans); % impute column-wise with mean of
        ↪ that column from the training set
    xTest = fillmissing(xTestRaw, 'constant', trainMeans);

    % 2- Z-score normalise using training statistics only
    mu = mean(xTrain, 1);
    sigma = std(xTrain, 0, 1);
    sigma(sigma == 0) = 1;

    xTrain = (xTrain - mu) ./ sigma;
    xTest = (xTest - mu) ./ sigma;
```

```
535
536
537     tic; % start timer
538
539     % Train NB with Gaussian distribution for continuous features
540     nbModel = fitcnb(xTrain, yTrain, 'DistributionNames', 'normal');
541     nbTrainTimes(fold) = toc; % stop timer
542
543     % Get predictions and posterior probabilities
544     [nbPred, nbPosterior] = predict(nbModel, xTest);
545     nbScores = nbPosterior(:, 2); % probability of class 1 (PCOS)
546
547     % Store for aggregate analysis
548     allNBPred = [allNBPred; nbPred];
549     allNBScores = [allNBScores; nbScores];
550     allNBTrue = [allNBTrue; yTest];
551
552     % Compute fold metrics
553     [acc, prec, rec, f1, spec] = ComputeMetrics(yTest, nbPred);
554
555     % Compute AUC
556     [~, ~, ~, auc] = perfcurve(yTest, nbScores, 1);
557
558     nbFoldMetrics.accuracy(fold) = acc;
559     nbFoldMetrics.precision(fold) = prec;
560     nbFoldMetrics.recall(fold) = rec;
561     nbFoldMetrics.f1(fold) = f1;
562     nbFoldMetrics.specificity(fold) = spec;
563     nbFoldMetrics.auc(fold) = auc;
564
565     fprintf('Acc=%.3f, F1=%.3f, AUC=%.3f\n', acc, f1, auc);
566 end
567
568 % Compute aggregate NB metrics
569 fprintf('\n--- Naive Bayes Results (Mean +/- Std) ---\n');
570 fprintf(' Accuracy: %.3f +/- %.3f\n', mean(nbFoldMetrics.accuracy), std(nbFoldMetrics.accuracy));
571 fprintf(' Precision: %.3f +/- %.3f\n', mean(nbFoldMetrics.precision),
        ↪ std(nbFoldMetrics.precision));
572 fprintf(' Recall:    %.3f +/- %.3f\n', mean(nbFoldMetrics.recall), std(nbFoldMetrics.recall));
573 fprintf(' F1-Score:  %.3f +/- %.3f\n', mean(nbFoldMetrics.f1), std(nbFoldMetrics.f1));
574 fprintf(' Specificity: %.3f +/- %.3f\n', mean(nbFoldMetrics.specificity),
        ↪ std(nbFoldMetrics.specificity));
575 fprintf(' AUC:       %.3f +/- %.3f\n', mean(nbFoldMetrics.auc), std(nbFoldMetrics.auc));
576
577 % Plot NB confusion matrix
578 figure('Name', 'NB Confusion Matrix', 'Position', [100, 100, 500, 450]);
579 PlotConfusionMatrix(allNBTrue, allNBPred, 'Naive Bayes Confusion Matrix');
580 saveas(gcf, 'fig_confusionNB.png');
581 fprintf('\nSaved fig_confusionNB.png\n');
582
583
584 %% (f) Performance Comparison and Limitations %%
585 fprintf('\n(f) Performance Comparison and Limitations...\n');
586
587 % Create comprehensive comparison table
588 fprintf('\n------------------------------------------------------------\n');
589 fprintf('                  MODEL COMPARISON\n');
590 fprintf('------------------------------------------------------------\n');
591 fprintf('%-15s | %-20s | %-20s | Winner\n', 'Metric', 'Neural Network', 'Naive Bayes');
```

```
fprintf('----------------------------------------------------------------\n');

% Calculate Brier score; lower is better
brierNN = mean((allNNScores - allNNTrue).^2); % mean-squared error between predicted probabilities and true labels; squaring punishes confident wrong predictions in an amplified manner
brierNB = mean((allNBScores - allNBTrue).^2);

metrics = {'Accuracy', 'Precision', 'Recall', 'F1-Score', 'Specificity', 'AUC', 'Brier Score'};
nnMeans = [mean(nnFoldMetrics.accuracy), mean(nnFoldMetrics.precision), ...
          mean(nnFoldMetrics.recall), mean(nnFoldMetrics.f1), ...
          mean(nnFoldMetrics.specificity), mean(nnFoldMetrics.auc), brierNN];
nnStds = [std(nnFoldMetrics.accuracy), std(nnFoldMetrics.precision), ...
         std(nnFoldMetrics.recall), std(nnFoldMetrics.f1), ...
         std(nnFoldMetrics.specificity), std(nnFoldMetrics.auc), 0];
nbMeans = [mean(nbFoldMetrics.accuracy), mean(nbFoldMetrics.precision), ...
          mean(nbFoldMetrics.recall), mean(nbFoldMetrics.f1), ...
          mean(nbFoldMetrics.specificity), mean(nbFoldMetrics.auc), brierNB];
nbStds = [std(nbFoldMetrics.accuracy), std(nbFoldMetrics.precision), ...
         std(nbFoldMetrics.recall), std(nbFoldMetrics.f1), ...
         std(nbFoldMetrics.specificity), std(nbFoldMetrics.auc), 0];

for i = 1:length(metrics) % for each metric determine winner by seeing which has higher mean (or lower for Brier score (prediction error))
    if strcmp(metrics{i}, 'Brier Score')
        if nnMeans(i) < nbMeans(i)
            winner = 'NN';
        elseif nbMeans(i) < nnMeans(i)
            winner = 'NB';
        else
            winner = 'Tie';
        end
        fprintf('%-15s | %.4f          | %.4f            | %s (lower=better)\n', ...
            metrics{i}, nnMeans(i), nbMeans(i), winner);
    else
        if nnMeans(i) > nbMeans(i)
            winner = 'NN';
        elseif nbMeans(i) > nnMeans(i)
            winner = 'NB';
        else
            winner = 'Tie';
        end
        fprintf('%-15s | %.3f +/- %.3f  | %.3f +/- %.3f    | %s\n', ...
            metrics{i}, nnMeans(i), nnStds(i), nbMeans(i), nbStds(i), winner);
    end
end
fprintf('----------------------------------------------------------------\n');

fprintf('\nInterpreting Brier Score:\n');
fprintf(' - Measures calibration, i.e., how well predicted probabilities match outcomes\n');
fprintf(' - Range: 0 (perfect) to 1 (worst)\n');
fprintf(' - Complements accuracy by assessing probability quality; not just the class\n');

fprintf('\nComparing Training Time:\n');
fprintf(' - NN: %.3f sec (total across %d folds)\n', sum(nnTrainTimes), optimalK);
fprintf(' - NB: %.3f sec (total across %d folds)\n', sum(nbTrainTimes), optimalK);
fprintf(' - NB is %.1fx faster than NN\n', sum(nnTrainTimes)/sum(nbTrainTimes));

% Plot ROC curves comparison
figure('Name', 'ROC Comparison', 'Position', [100, 100, 600, 500]);
```

```
hold on;
[xNN, yNN, ~, aucNN] = perfcurve(allNNTrue, allNNScores, 1); % the probability that a
    ↪ random-chosen positive sample scores higher than a random-chosen negative sample
[xNB, yNB, ~, aucNB] = perfcurve(allNBTrue, allNBScores, 1);
plot(xNN, yNN, 'b-', 'LineWidth', 2);
plot(xNB, yNB, 'r--', 'LineWidth', 2);
plot([0, 1], [0, 1], 'k:', 'LineWidth', 1);
hold off;
xlabel('False Positive Rate');
ylabel('True Positive Rate');
title('ROC Curve Comparison');
legend(sprintf('Neural Network (AUC=%.3f)', mean(nnFoldMetrics.auc)), ...
      sprintf('Naive Bayes (AUC=%.3f)', mean(nbFoldMetrics.auc)), ...
      'Random Classifier', 'Location', 'southeast');
grid on;
saveas(gcf, 'fig_rocComparison.png');
fprintf('\nSaved: fig_rocComparison.png\n');

% Plot metrics comparison bar chart
figure('Name', 'Metrics Comparison', 'Position', [100, 100, 800, 500]);
x = 1:length(metrics);
width = 0.35;
hold on;
bar(x - width/2, nnMeans, width, 'FaceColor', [0.3, 0.5, 0.8]);
bar(x + width/2, nbMeans, width, 'FaceColor', [0.8, 0.4, 0.3]);
errorbar(x - width/2, nnMeans, nnStds, 'k.', 'LineWidth', 1);
errorbar(x + width/2, nbMeans, nbStds, 'k.', 'LineWidth', 1);
hold off;
set(gca, 'XTick', x, 'XTickLabel', metrics);
ylabel('Score');
title('Classification Metrics Comparison');
legend('Neural Network', 'Naive Bayes', 'Location', 'southwest');
ylim([0, 1.1]);
grid on;
saveas(gcf, 'fig_metricsComparison.png');
fprintf('Saved: fig_metricsComparison.png\n');

% Side-by-side confusion matrices
figure('Name', 'Confusion Matrix Comparison', 'Position', [100, 100, 1000, 450]);
subplot(1, 2, 1);
PlotConfusionMatrix(allNNTrue, allNNPred, 'Neural Network');
subplot(1, 2, 2);
PlotConfusionMatrix(allNBTrue, allNBPred, 'Naive Bayes');
saveas(gcf, 'fig_confusionComparison.png');
fprintf('Saved: fig_confusionComparison.png\n');

%% Correlation Heatmap for Independence Assumption Analysis
figure('Name', 'Feature Correlation Matrix', 'Position', [100, 100, 900, 800]);
corrMatrix = corr(data, 'rows', 'pairwise');
imagesc(corrMatrix);
colormap(redblue(256));
colorbar;
caxis([-1, 1]);
title('Feature Correlation Matrix');
set(gca, 'XTick', [], 'YTick', []);
xlabel('Features (1-41)');
ylabel('Features (1-41)');
saveas(gcf, 'fig_correlationMatrix.png');
fprintf('Saved: fig_correlationMatrix.png\n');
```

```
707
708 %% Save Outputs %%
709
710 fprintf('\n-------------\n');
711 fprintf('OUTPUT SUMMARY\n');
712 fprintf('\n-------------\n');
713
714 % Export metrics to CSV
715 resultsTable = table(metrics', nnMeans', nnStds', nbMeans', nbStds', ...
716     'VariableNames', {'Metric', 'NN_Mean', 'NN_Std', 'NB_Mean', 'NB_Std'});
717 writetable(resultsTable, 'classification_results.csv');
718 fprintf('Saved: classification_results.csv\n');
719
720 % Print final summary
721 fprintf('\nFigures generated:\n');
722 fprintf(' - fig_classDistribution.png\n');
723 fprintf(' - fig_kfoldComparison.png\n');
724 fprintf(' - fig_featureDistributions.png\n');
725 fprintf(' - fig_confusionNN.png\n');
726 fprintf(' - fig_confusionNB.png\n');
727 fprintf(' - fig_rocComparison.png\n');
728 fprintf(' - fig_metricsComparison.png\n');
729 fprintf(' - fig_confusionComparison.png\n');
730 fprintf(' - fig_correlationMatrix.png\n');
731
732 fprintf('\nTask-4 classification completed successfully.\n');
733
734 %% Helper Functions %%
735
736 function c = redblue(m)
737     %Diverging colourmap: blue (negative) -> white (zero) -> red (positive)
738     if nargin < 1, m = 256; end
739     half = floor(m/2);
740     r = [linspace(0,1,half), ones(1,m-half)]';
741     g = [linspace(0,1,half), linspace(1,0,m-half)]';
742     b = [ones(1,half), linspace(1,0,m-half)]';
743     c = [r, g, b];
744 end
745
746 function [acc, prec, rec, f1, spec] = ComputeMetrics(trueLabels, predictions)
747     %Compute the classification metrics based on true labels and predictions
748     % Input: trueLabels (ground truth), predictions (binary)
749     % Output: accuracy, precision, recall, f1Score, specificity
750
751     % Confusion matrix elements
752     tp = sum(predictions == 1 & trueLabels == 1);
753     tn = sum(predictions == 0 & trueLabels == 0);
754     fp = sum(predictions == 1 & trueLabels == 0);
755     fn = sum(predictions == 0 & trueLabels == 1);
756
757     % Metrics calculation with zero-division handling
758     acc = (tp + tn) / (tp + tn + fp + fn);
759
760     % The if-elses prevent division by zero thus crashes
761
762     if (tp + fp) > 0
763         prec = tp / (tp + fp);
764     else
765         prec = 0;
```

```
    end

    if (tp + fn) > 0
        rec = tp / (tp + fn);
    else
        rec = 0;
    end

    if (prec + rec) > 0
        f1 = 2 * (prec * rec) / (prec + rec);
    else
        f1 = 0;
    end

    if (tn + fp) > 0
        spec = tn / (tn + fp);
    else
        spec = 0;
    end
end

function PlotConfusionMatrix(trueLabels, predictions, titleStr)
    %Generate and display confusion matrix figure
    % Input: trueLabels (ground truth); predictions (binary); titleStr (title)

    % Compute confusion matrix (1=true; 0=false; '==' = IF equal)
    tp = sum(predictions == 1 & trueLabels == 1); % true positives
    tn = sum(predictions == 0 & trueLabels == 0); % true negatives
    fp = sum(predictions == 1 & trueLabels == 0); % false positives
    fn = sum(predictions == 0 & trueLabels == 1); % false negatives

    confMat = [tn, fp; fn, tp];

    % Plot confusion matrix wherein rows = actual and columns = predicted
    imagesc(confMat);
    colormap(flipud(hot));
    colorbar;

    % Text annotations
    textStrings = num2str(confMat(:));
    [x, y] = meshgrid(1:2, 1:2);
    text(x(:), y(:), textStrings, 'HorizontalAlignment', 'center', ...
        'FontSize', 14, 'FontWeight', 'bold');

    % Labels (maps matrix structure to axes)
    set(gca, 'XTick', [1, 2], 'XTickLabel', {'Predicted 0', 'Predicted 1'}); % columms; predictions
    set(gca, 'YTick', [1, 2], 'YTickLabel', {'Actual 0', 'Actual 1'}); % rows; actuals
    xlabel('Predicted Class');
    ylabel('Actual Class');
    title(titleStr);

    % Accuracy annotation
    acc = (tp + tn) / (tp + tn + fp + fn); % compute accuracy: how often is the model right
    annotation('textbox', [0.15, 0.02, 0.7, 0.05], 'String', ...
        sprintf('Accuracy: %.1f%%', acc*100), 'HorizontalAlignment', 'center', ...
        'FontSize', 10, 'EdgeColor', 'none');
end
```

Appendix B: 5-Minute Video Demo

- YouTube link: https://youtu.be/aVlyZZQZO6M

Appendix C: AI Declaration

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.
	A11 - Other uses not listed above Please specify:
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:

Figure 9: Student Declaration of AI Tool use in this Assessment Table

I declare that I've used the AI tools listed below whilst preparing this assessment. I've read and understood the University of Plymouth's policy on the use of AI tools in assessment and confirm that my use falls within

the coursework's allowed categories, i.e. **A2 (Planning and Structuring Projects)** and **A4 (Research Assistance)**.

AI Tool Used	**Purpose of Use**	**Extent of Use**
ChatGPT	Finding relevant pages to read in the paper **(A4)**	Few times if the paper is too long
ChatGPT	General conversations via web-search AI about prevalent papers to read about how the topics relates to others' studies **(A4)**	Few times at the end

☒ I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.

☒ I confirm that all details provide above are an accurate description of how AI was used for this assessment.

Date: _____